

Autonomes Anlegen einer chirurgischen Naht mit kontinuierlichen Trajektorien im geschlossenen Regelkreis

vorgelegt von

Heubo Heubo Franck Junior

EDV.Nr.:928 238

dem Fachbereich VI – Technische Informatik– Medieninformatik – Data Science

der Berliner Hochschule für Technik Berlin

vorgelegte Bachelorarbeit

zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Technische Informatik

Tag der Abgabe 18. August 2025



Gutachter

Prof. Dr.-Ing.Francisco Morales Serrano

Prof. Dr.-Ing.Wolfgang Kessler

Berliner Hochschule für Technik

Berliner Hochschule für Technik

Kurzfassung

Liebe Leserinnen und Leser,
herzlich willkommen zu meiner Bachelorarbeit, die sich mit dem spannenden Thema des autonomen Anlegens einer chirurgischen Naht mit kontinuierlichen Trajektorien im geschlossenen Regelkreis befasst.

Ziel dieser Bachelorarbeit ist die Optimierung des Bewegungsverhaltens von chirurgischen Robotersystemen, das beim automatisierten Nähen innerhalb einer Wunde zum Einsatz kommt. Der aktuelle Stand des Systems weist eine diskontinuierliche Bewegung der Roboterarme während des Nahtvorgangs auf, was die Präzision und Effizienz des Eingriffs beeinträchtigen kann. Im Rahmen dieser Arbeit wird eine Methode entwickelt, um eine kontinuierliche und gleichmäßige Bewegung zu ermöglichen.

Ein weiterer wesentlicher Bestandteil der Arbeit ist die visuelle Erkennung der Nadelspitze, sobald diese nach dem Durchdringen des Gewebes aus der Wunde austritt. Hierzu wird ein bildverarbeitungsgestütztes System implementiert, das mithilfe einer Kamera die exakte Position der Nadel erfasst. Diese Information wird anschließend in Echtzeit an einen zweiten Roboterarm übermittelt, der daraufhin präzise auf die erkannte Position der Nadelspitze zugreifen kann. Am Ende wird mit demselben Verfahren auch die exakte Position des Fadens ermittelt und an den ersten Roboterarm übergeben, damit dieser den Knoten präzise schließen kann.

Durch die Kombination aus Trajektorienoptimierung mit visueller Nadel und Faden Verfolgung soll die Gesamtleistung des Robotersystems im chirurgischen Kontext verbessert und ein wichtiger Schritt in Richtung einer automatisierten, sicheren und zuverlässigen Roboterassistenz bei Nahtprozessen erreicht werden.

Abstract

The goal of this bachelor thesis is to optimise the movement behaviour of a surgical robot system that is used for automated suturing within a wound. The current state of the system shows discontinuous movement of the robot arms during the suturing process, which can impair the precision and efficiency of the procedure. As part of this thesis, a method is being developed to smooth the robot's trajectory and enable continuous and smooth movement. Another essential component of the thesis is the visual recognition of the needle tip as soon as it emerges from the wound after penetrating the tissue. For this purpose, an image processing-based system is implemented that uses a camera to detect the exact position of the needle. This information is then transmitted in real time to a second robot arm, which can then precisely access the detected position of the needle tip. By combining trajectory optimisation and visual needle tracking, the overall performance of the robot system in a surgical context is to be improved and an important step towards The combination of trajectory optimisation and visual needle tracking is intended to improve the overall performance of the robot system in a surgical context and represent an important step towards automated, safe and reliable robot assistance in suturing processes

Vorwort: Persönliches Motiv

Im Verlauf meines Bachelorstudiums entwickelte ich ein besonderes Interesse für die Fachgebiete Regelungstechnik, Robotik und Mathematik. Diese Disziplinen zeichnen sich nicht nur durch ihre hohe theoretische Tiefe aus, sondern auch durch ihre vielseitigen Anwendungsmöglichkeiten insbesondere im Bereich der Automatisierung und intelligenten Systeme.

Deswegen war es für mich besser, meine Bachelorarbeit in einem Themenfeld zu schreiben, das diese drei Interessen vereint. In Abstimmung mit meinem betreuenden Dozenten, Herrn Prof. Morales, ergab sich nach intensiven Gesprächen das Thema **Anlegen einer chirurgischen Naht mit kontinuierlichen Trajektorien im geschlossenen Regelkreis**. Dieses interdisziplinäre Projekt verbindet technische Fragestellungen aus der Robotik mit medizinischen Anwendungen, einem Bereich, in dem technologische Innovationen einen unmittelbaren gesellschaftlichen Nutzen entfalten können.

Ziel der Arbeit ist es, das Bewegungsverhalten eines chirurgischen Roboters zu optimieren und ihm ein höheres Maß an Autonomie zu verleihen. Die bisherige Steuerungsmethode war für die notwendigen Anforderungen insbesondere im Hinblick auf Präzision und Kontinuität nicht ausreichend. Daher habe einen neuen Lösungsansatz entwickelt, der sich in zwei zentrale Teilbereiche gliedert:

- **Trajektorienoptimierung:** Durch die Definition einer elliptischen Bahn für die Nadelspitze soll die Bewegung innerhalb der Wunde kontinuierlich, gleichmäßig und möglichst gewebeschonend erfolgen.
- **Visuelle Objekterkennung:** Mithilfe einer externen Kamera wird die Position der Nadelspitze erfasst, sobald sie aus dem Gewebe herauskommt. Ebenso wird die Position des Fadens für die Erstellung Knoten erkannt. Die ermittelten Informationen dienen dazu, einen zweiten Roboterarm präzise zur erfassten Nadelspitze bzw. Fadenposition zu führen. Dieser Ansatz wird als *Visual Servoing* bezeichnet.

Die Bearbeitung dieses Themas ermöglichte mir nicht nur eine vertiefte Auseinandersetzung mit modernen Regelungs- und Bildverarbeitungsverfahren, sondern eröffnete mir auch einen praxisnahen Einblick in die Anwendung von Robotik im medizinischen Umfeld, ein Bereich, dessen gesellschaftliche Relevanz mich besonders motiviert.

Danksagung

An dieser Stelle möchte ich allen danken, die mich während der Arbeit an dieser Bachelorarbeit unterstützt und begleitet haben.

Zunächst gilt mein besonderer Dank Herrn Prof. Dr. Francisco Morales Serrano, meinem Betreuer, für die fachliche Unterstützung, die wertvollen Ratschläge und die konstruktive Kritik während der gesamten Arbeit. Seine Expertise und Geduld haben maßgeblich zum Gelingen dieser Arbeit beigetragen.

Ebenso danke ich Herrn Prof. Dr. Wolfgang Kessler, dem Gutachter dieser Arbeit, für seine Zeit und das Interesse an meinem Projekt.

Des Weiteren danke ich Herrn Heiko, Laboringenieur an der Berliner Hochschule für Technik, für seine freundliche Unterstützung bei der Lösung von Netzwerkproblemen im Labor. Mein besonderer Dank gilt auch Herrn Philip Grefoll und Frau Lama Ali, die mir mit hilfreichen Hinweisen zur Seite standen und mich bei der Durchführung der Messungen tatkräftig unterstützt haben. Ihre Hilfsbereitschaft und fachliche Unterstützung haben wesentlich zum Gelingen dieser Arbeit beigetragen.

Ein herzlicher Dank geht auch an meine Familie und meine Freunde, die mich während der gesamten Studienzeit, im besonderen während der Anfertigung dieser Arbeit unterstützt und motiviert haben. Ihr Rückhalt und ihre Geduld haben mir in schwierigen Phasen immer wieder Kraft gegeben.

Besonderer Dank gilt auch meinen Kommilitonen und Kollegen, mit denen ich viele Diskussionen führen und Ideen austauschen konnte. Dieser Austausch hat mich sowohl fachlich als auch persönlich weitergebracht.

Schließlich möchte ich mich bei allen bedanken, die direkt oder indirekt zum Gelingen dieser Arbeit beigetragen haben. Ohne ihre Unterstützung wäre diese Arbeit nicht möglich gewesen.

Erklärung

Ich versichere, dass ich diese Abschlussarbeit ohne fremde Hilfe selbstständig verfasst und nur die angegebenen Quellen und Hilfsmittel benutzt habe. Ich habe für die sprachliche und grammatikalische Korrektur unterstützende KI-Tools verwendet, darunter ChatGPT, Copilot, Gemini, DeepL Write, DeepL Translator und Grammarly. Wörtlich oder dem Sinn nach aus anderen Werken entnommene Stellen sind unter Angabe der Quellen kenntlich gemacht.

Datum

Unterschrift

Inhaltsverzeichnis

1	Einleitung	3
1.1	Hintergrund	3
1.2	Beschreibung des Ist-Zustands	3
1.3	Problemstellung und Zielsetzung	5
1.4	Bedeutung der Visual Servoring in der Robotik	6
2	Theoretischer Hintergrund und Literaturrecherche	7
2.1	Universal Robots UR3e	7
2.2	Vorstellung des uEye Kamera	13
2.3	Technische Grundladgen von Visual Servoring und andere Methode	14
3	Methodik	19
3.1	Entwurf und Konzeption der kontinuierlichen Bewegung der Nadelspitze innerhalb der Wunde	19
3.2	Entwurf und Umsetzung der Bildverarbeitung zur Erkennung von Nadel und Faden	25
3.2.1	Vorgehensweise	25
3.2.2	Lochkameramodell	26
3.2.3	Kalibrierung des uEye Kamera	27
3.2.4	Berechnung der Transformationsmatrix	29
3.3	Beschreibung des experimentellen Aufbaus und der verwendeten Hardware	32
3.4	Realisierung	34
3.4.1	Realisierung der elliptische Bewegung	35
3.4.2	Realisierung der Bildverarbeitung zur Erkennung von Nadel und Faden	36
4	Testmethoden und Ergebnisse	43
4.1	Kontinuierliche elliptische Bewegung: Testmethodik und Ergebnisse	43
4.2	IBVS-Nadelerkennung: Testmethodik und Ergebnisse	44
5	Diskussion	49
5.1	Diskussion der Ergebnisse	49
5.2	Lösungsansätze und Weiterentwicklung	50
6	Fazit	51

Abbildungsverzeichnis

1.1	Diskontinuierliche Bewegung der Nadelspitze innerhalb der Wunde.	4
1.2	RoboterArm 2 trifft nicht den Nadelspitze.	5
2.1	Der UR3e Roboter[7]	8
2.2	3D Arbeitsraum des UR3e-Roboters	9
2.3	2D Arbeitsraum des UR3e-Roboters	9
2.4	Darstellung des UR3e-Roboters in zwei Perspektiven [10]	10
2.5	Robotiq Hand-E Greifer [13]	12
2.6	Die Verbindung zwischen dem Greifer und seiner Kupplung[13]	13
2.7	Mechanische Spezifikationen der uEye Kamera [24]	14
2.8	PBVS Architecture[15]	16
2.9	IVBS Architecture [15]	16
3.1	Elliptische Kurven [16]	20
3.2	Ellipse in ein Koordinatensystem	20
3.3	TCP des Roboters und Nadelspitze	22
3.4	Blocksystem	23
3.5	Position des TCP und der Nadelspitze im Global Koordinatensystem	23
3.6	Formelsimulation des Vektors vom TCP zur Nadelspitze	24
3.7	Formelsimulation der Bewegung	25
3.8	Erkennung der Nadel mithilfe des IBVS-Modells	26
3.9	Erkennung des Fadens mithilfe des IBVS-Modells	26
3.10	Lochkameramodell [19]	27
3.11	Position des globalen Koordinatensystems vs. des Kamerakoordinatensystems	30
3.12	Grundaufbau des Versuch: a) Roboteraufhängung des experimentellen Grundaufbaus mit verstellbarem Operationstisch im Vordergrund. b) uEye Kamera an oberer Querstrebe, aufgenommen im Labor für autonome Chirurgie an der Berliner Hochschule für Technik	32
3.13	uEye CP Kamera	33
3.14	50/60 Hz LED Lichtquelle	33
3.15	Messmaterialien und Schachbrettmuster	34
3.16	Implementierung der elliptischen Bewegung	35
3.17	Kalibrierungsprogramm	36
3.18	Code für automatische Aufnahme eines Bildes	37
3.19	Transformationsmatrix	38
3.20	Funktion zur Nadeldetektion	39
3.21	Funktion zur Automatisierung des Prozesses und Steuerung des Roboters	40
3.22	Funktion zur Fadendetektion	40
3.23	Funktion zur Automatisierung des Prozesses und Steuerung des Roboters	41
4.1	Ergebnisse der Visual-Servoing-Prüfung des linken Arms in 100 Testläufen	44

4.2	Ergebnisse der Visual-Servoing-Prüfung des linken Arms in 100 Testläufen . . .	45
4.3	Erkennung der Nadel und des Fadens	46
4.4	Visual-Servoing: Ergebnisse beim Nadelzugriff in 50 Testläufen	46
4.5	Visual-Servoing: Ergebnisse beim Fadenzugriff in 50 Testläufen	47

Tabellenverzeichnis

2.1	Rotationseigenschaften der Gelenke des UR3e-Roboters [8]	8
2.2	DH-Parameter für die Gelenke des Roboters [10]	10
2.3	Vergleich zwischen PBVS und IBVS	17
4.1	Testtabelle der elliptischen Bewegung und Ergebnisse	43

Abkürzungsverzeichnis

UR	Universal Robots
DoF	Degree of Freedom (Freiheitsgrad)
DH-Methode	Denavit-Hartenberg-Methode
TCP	Tool Center Point
I/O	Input/Output
IBVS	Image-Based Visual Servoing
PBVS	Position-Based Visual Servoing
$\mathbf{P}_t$	Position der Nadelspitze
$\mathbf{P}_{tcp}$	Position des TCP
$\mathbf{O}_{tcp}$	Orientierung des TCP
RTU	Remote Terminal Unit
GND	Ground
RGB	Red Green Blue(Farbraum)
SDK	Software Developemet Kit
IDS	Imaging Development Systems
LED	Leuchtdioden
ROI	Region of Interest (Interessensbereich)

Kapitel 1

Einleitung

1.1 Hintergrund

in den Vergangenen Jahrzehnten hat sich die Robotik rasant weiterentwickelt und ist heutzutage ein integraler Bestandteil zahlreicher Branchen und Lebensbereiche. Sie umfasst interdisziplinäre Bereiche wie Medizin, Mechanik, Elektronik, Informatik und künstliche Intelligenz. Mechanischen Komponenten ermöglichen die Konstruktion der physischen Strukturen und Bewegungsfähigkeit, während elektronische Bauteile wie Sensoren, Aktoren und Steuerungen die Grundlage für präzise Bewegungsabläufe schaffen. Informatik und künstliche Intelligenz spielen eine entscheidende Rolle bei der Entwicklung von Entscheidungsalgorithmen, der Mustererkennung und dem maschinellen Lernen. Gleichzeitig sorgt die Steuerungs- und Regelungstechnik für die präzise Ausführung gewünschter Aktionen. Weitere zentrale Aspekte der Robotik sind die Wahrnehmung und Sensorik, die Mensch-Maschine-Kommunikation sowie die Automatisierung, die in der Industrie und im Alltag zahlreiche Prozesse erleichtern[1].

1.2 Beschreibung des Ist-Zustands

Ein Teil der Steuerung des Roboters für eine bestimmte Aufgabe erfolgt manuell. In einem anderen Bereich zeigt der Roboter jedoch ein unerwünschtes Verhalten.

Innerhalb der Wunde ist das Bewegungsverhalten des Roboters nicht ideal, da er sich dort nur diskontinuierlich, also in ruckartigen Bewegungen, fortbewegt. Erwünscht ist jedoch eine kontinuierliche, flüssige Bewegung, um präziser und schonender arbeiten zu können.

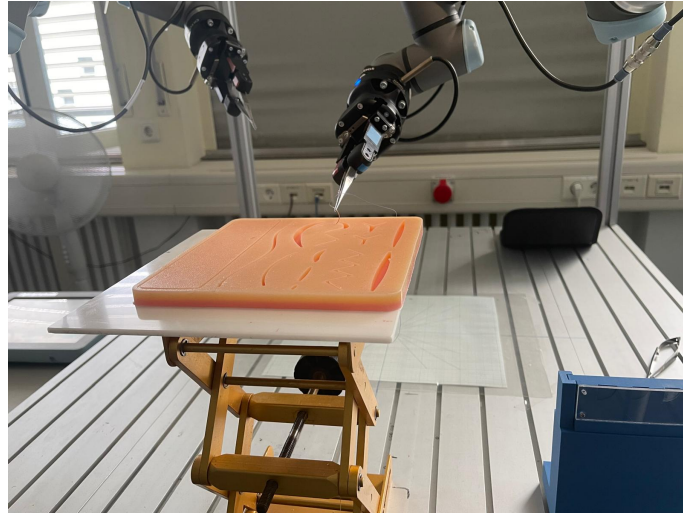


Abbildung 1.1: Diskontinuierliche Bewegung der Nadelspitze innerhalb der Wunde.

Die Abbildung 1.1 zeigt ein Bild der diskontinuierlichen Bewegung der Nadel in der Wunde. Diese Bewegung erfolgt in mehreren Schritten.

- erstmal bewegt der Roboter sich den allgemeinen Bereich der Wunde, um das Risiko von Singularitäten zu minimieren
- Im ersten Schritt bewegt sich der TCP des Roboters entlang der z-Achse senkrecht in die Wunde hinein. Diese Bewegung dient dazu, das chirurgische Werkzeug (z.B. eine Nadelhalterspitze oder ein Pad) so weit abzusenken, dass es sicher in das Gewebe eindringen kann. Die Eindringtiefe wird durch den Parameter δ_z definiert. Der TCP wird dabei um den Vektor $\begin{pmatrix} 0 \\ 0 \\ \delta_z \end{pmatrix}$ verschoben
- Dann erfolgt eine Drehbewegung des Werkzeugs um die Nadelspitze. Diese Rotation wird nach dem senkrechten Eindringen der Nadelspitze durchgeführt, damit sie weiter entlang der gewünschten Richtung geführt wird. Die Nadelspitze bleibt dabei der feste Drehpunkt. Der TCP des Roboters wird Schritt für Schritt um einen kleinen Winkel δ_θ gedreht. Dadurch entsteht eine kreisförmige Bewegung, die durch eine feste Anzahl n an diskreten Drehschritten beschrieben wird.
- Weiterhin erfolgt eine Translation des TCP in y-Richtung, also seitlich entlang der Gewebeöffnung. Ziel ist es, dass die Nadelspitze die gegenüberliegende (rechte) Wundkante durchdringt. Diese Bewegung wird benötigt, um den Nahtvorgang auf der anderen Seite fortzusetzen.

Der TCP wird dabei um den Betrag δ_y in y-Richtung verschoben $\begin{pmatrix} 0 \\ \delta_y \\ 0 \end{pmatrix}$, während die Orientierung der Nadel α_i unverändert bleibt ($\alpha_{i+1} = \alpha_i$)

- Im nächsten Schritt wird die Nadel um ihre eigene Spitze rotiert. Dabei bleibt die Spitze der Nadel unverändert an ihrer Position, während sich die restliche Nadel um einen bestimmten Winkel δ_θ dreht.

Ziel dieser Bewegung ist es, die Nadel für den weiteren Nahtverlauf korrekt auszurichten. Durch die Rotation verändert sich die Orientierung der Nadel ($\alpha_{i+1} = \alpha_i + \delta_\theta$), während die Position der Nadelspitze konstant bleibt.

Im nächsten Schritt wird die Nadel um ihren geometrischen Mittelpunkt rotiert. Der Mittelpunkt bleibt also konstant, die Nadel dreht sich jedoch um einen bestimmten Winkel weiter. Der Zweck dieser Bewegung besteht darin, die Nadelspitze durch die Gewebegewebeschichtung auf die gegenüberliegende Seite zu führen und die Naht dort fortzusetzen. Ein der Rotation entsprechender Kreis wird berücksichtigt, mit dem Rotationszentrum auf der Nadel.

Ein weiteres Problem entsteht, wenn die Nadel aus der Wunde herauskommt: In diesem Moment muss ihre Position manuell an den zweiten Roboterarm übergeben werden, damit dieser die Nadel ergreifen kann. Das ist jedoch nicht immer einfach, denn die exakte Position der Nadelspitze kann sich minimal verändern, zum Beispiel durch Wind, kleine Bewegungen oder andere äußere Einflüsse. Diese Abweichungen liegen oft im Millimeterbereich, reichen aber aus, dass der Greifer des zweiten Roboterarms die Nadelspitze nicht exakt trifft.

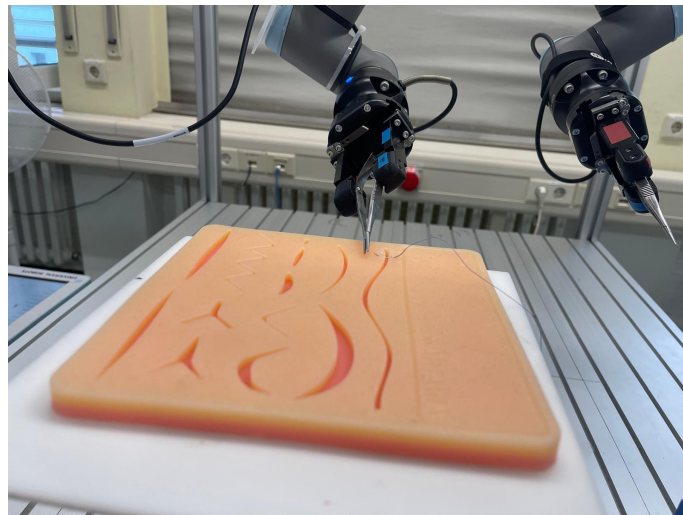


Abbildung 1.2: RoboterArm 2 trifft nicht den Nadelspitze.

1.3 Problemstellung und Zielsetzung

Obwohl das aktuelle System funktionsfähig ist, weist es mehrere Schwächen auf. Zunächst verläuft die Roboterbewegung innerhalb der Wundschale geruckelt und stückweise, was für die präzise chirurgische Anwendung nicht optimal ist. Zweitens erfordert die Übergabe der Nadel an den zweiten Arm des Roboters immer eine manuelle Änderung, da selbst die geringste Positionsänderung dazu führen kann, dass der Roboter die Nadel nicht ordnungsgemäß greift.

Ziel dieser Arbeit ist es, die beschriebenen Schwächen zu überwinden und eine gleichmäßige sowie präzise Steuerung des Roboters zu ermöglichen. Dafür werden folgende zwei Hauptproblematiken bearbeitet:

- Implementierung einer kontinuierlichen elliptischen Bewegungsbahn, um eine gleichmäßige Führung der Nadelspitze während des Nahtvorgangs sicherzustellen
- Einsatz eines kamerabasierten Visual-Servoing-Systems, zur automatischen Erkennung der Nadelposition und zur präzisen Übergabe an den Greifarm, einschließlich der Erkennung des Fadens, um anschließend den Knoten zu erstellen.

1.4 Bedeutung der Visual Servoring in der Robotik

Ein wichtiger Forschungsbereich in der Robotik ist die visuelle Kontrolle von Robotern. Dabei kommt ein integriertes Kamerasystem zum Einsatz, um die Bewegungsabläufe und die Position von Objekten zu erfassen, zu analysieren und über Verfahren der Objektdetektion zu identifizieren. Auf diese Weise können Roboter die Bewegung eines erkannten Objekts beobachten, verfolgen und sich daran ausrichten. Nach der Verarbeitung der visuellen Daten sind sie in der Lage, die erfassten Bewegungsabläufe präzise nachzuvollziehen und gezielt auszuführen.

Roboter, die von Videobildern gesteuert werden, sind vielseitig einsetzbar und weisen in der Chirurgie signifikante Vorteile auf [2]. Die Verwendung einer integrierten Kamera ermöglicht die genaue Steuerung des Roboters und die präzise Durchführung der kleinsten chirurgischen Eingriffe. Darüber hinaus wird die Bildgebung des Operationsfeldes in den meisten Fällen sichtbar verbessert, was dazu beiträgt, dass Diagnosen genauer gestellt und krankhafte Veränderungen zuverlässiger erkannt werden.

Ein weiteres Anwendungsgebiet, in dem visuell gesteuerte Roboter aktiv eingesetzt werden, ist die Unterhaltungsbranche. Diese Roboter gewähren breite Möglichkeiten ihrer Nutzung: Sie können nach Tanzchoreografien tanzen und Bewegungen aus Sportarten ausführen, was neuartige interaktive Unterhaltungserlebnisse schafft. So wurde beispielsweise die Baureihe QRIO von Sony in Japan entwickelt: Mehrere Roboter können gleichzeitig mit Menschen tanzen. QRIO folgt einem speziellen vorprogrammierten Tanzmuster und kann auf der Grundlage seines kamerabasierten visuellen Sensorsystems mit Menschen interagieren. QRIO ist eines der ersten Beispiele für den kreativen Einsatz von Robotik und Maschinenbau für die interaktive Unterhaltungsindustrie.[3]

Roboter, die visuell gesteuert werden, sind in der Industrie ebenfalls immer gefragter. So werden sie beispielsweise dafür verwendet, mit ihren Kameras Objekte zu erfassen und durch präzise Bewegungen an bestimmte Orte zu verschieben, was die Automatisierung weit effizienter gestaltet und vor allem ausgedehntere Produktionsraten ermöglicht. Zusammenarbeit zwischen Mensch und Roboter gehören ebenfalls zu den Verbesserungen. Infolgedessen ist nicht nur eine höhere Produktivität und Qualität der Prozesse, sondern auch eine Verbesserung in Bezug auf die Sicherheitsstandards zu erreichen. Geeignete Kontrollmethoden und Metriken sind ebenfalls hierfür verfügbar.[4]

Kapitel 2

Theoretischer Hintergrund und Literaturrecherche

In diesem Kapitel werden die in dieser Arbeit eingesetzten Hardwarekomponenten des Roboterarm UR3e von Universal Robots sowie die IDS uEye Kamera ausführlich vorgestellt und analysiert.

Neben der technischen Beschreibung beider Systeme erfolgt auch eine Einordnung in den wissenschaftlichen Kontext, indem relevante Forschungsarbeiten und vergleichbare Ansätze aus der Literatur betrachtet werden. Ziel ist es, die Eignung und die spezifischen Eigenschaften der ausgewählten Geräte im Hinblick auf die geplante Anwendung zu bewerten.

Diese Analyse soll einen umfassenden Überblick bieten und als fundierte Grundlage für die anschließende praktische Umsetzung dienen.

2.1 Universal Robots UR3e

Der UR3e des dänischen Roboterherstellers Universal Robots ist ein kollaborativer Tischroboterarm mit sehr geringen Abmessungen. Er wird normalerweise in engen Arbeitsbereichen und bei der Automatisierung komplexer und präziser Arbeitszyklen eingesetzt. Der UR3e hat eine Reichweite von 500 mm, kann Objekte mit einer maximalen Traglast von 3 kg greifen und ist mit sechs Rotationsgelenken und Achsen ausgestattet.

Dank seines geringen Gewichts von nur 11,2 kg und eines Basisdurchmessers von 128 mm lässt sich der Roboterarm einfach in die unterschiedlichsten Arbeitsbereiche integrieren [5].

Ein herausragendes Merkmal des UR3e ist seine Wiederholgenauigkeit von $\pm 0,03$ mm. Das bedeutet, dass der Roboter in der Lage ist, eine zuvor angefahrne Position mit sehr hoher Präzision erneut zu erreichen. Diese Genauigkeit ist insbesondere für präzise Montage-, Handhabungs- und Positionieraufgaben von großer Bedeutung [6].



Abbildung 2.1: Der UR3e Roboter[7]

Der Roboter verfügt über sechs Freiheitsgrade (DoF), die durch aktiv angetriebene Rotationsgelenke ermöglicht werden. Jedes dieser Gelenke lässt sich unabhängig von den anderen bewegen, was eine exakte Steuerung sowie eine hohe Bewegungsfreiheit und Anpassungsfähigkeit ermöglicht. Die spezifischen Drehbereiche und maximalen Rotationsgeschwindigkeiten der einzelnen Achsen sind in der nachfolgenden Tabelle zusammengefasst .

Bezeichnung	Technischer Ausdruck	Arbeitsraum	Max. Geschwindigkeit
Basisgelenk	Joint 1	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
Schultergelenk	Joint 2	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
Ellbogengelenk	Joint 3	$\pm 360^\circ$	$\pm 180^\circ/\text{s}$
1. Handgelenk	Joint 4	$\pm 360^\circ$	$\pm 360^\circ/\text{s}$
2. Handgelenk	Joint 5	$\pm 360^\circ$	$\pm 360^\circ/\text{s}$
3. Handgelenk	Joint 6	unbegrenzt	$\pm 360^\circ/\text{s}$

Tabelle 2.1: Rotationseigenschaften der Gelenke des UR3e-Roboters [8]

Im folgenden dann die sich aus dieser Tabelle ergebenden 3D und 2D Arbeitsräume des UR3e Roboters.

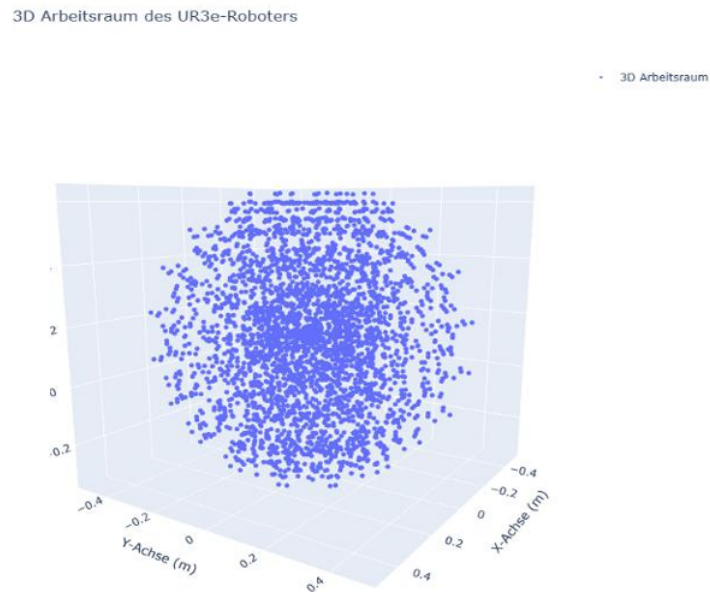


Abbildung 2.2: 3D Arbeitsraum des UR3e-Roboters

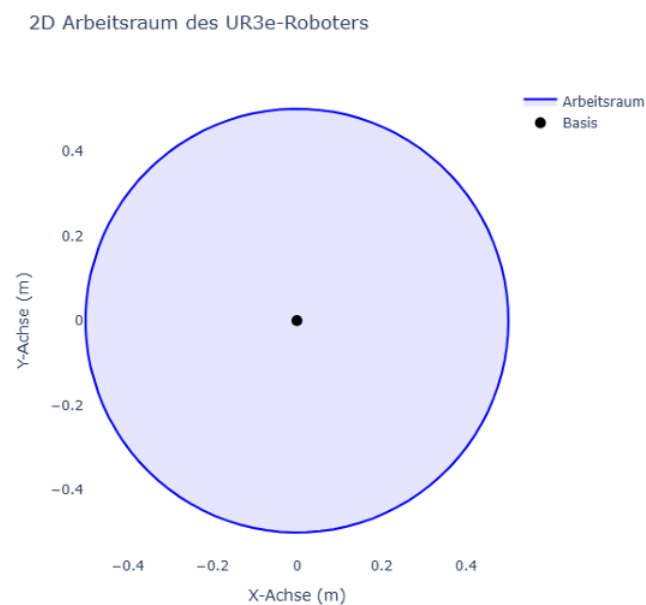


Abbildung 2.3: 2D Arbeitsraum des UR3e-Roboters

Die Kinematik des Roboters wird mithilfe der Denavit-Hartenberg-Methode (DH Methode) analysiert. Dabei berücksichtigt die Methode, dass die Position jedes Gelenks J_k von der Position des vorherigen Gelenks J_{k-1} beeinflusst wird [9]. Dazu wird der Roboter in eine günstige Position gebracht, um die kinematischen Parameter zu definieren und das Modell zu erstellen.

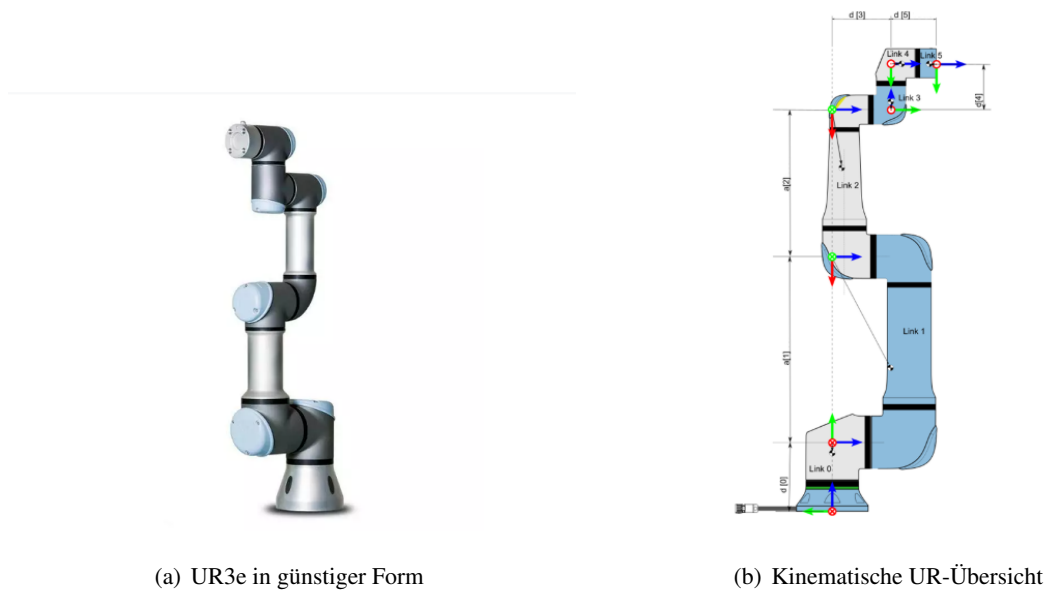


Abbildung 2.4: Darstellung des UR3e-Roboters in zwei Perspektiven [10]

Gelenk i	a_i	α_i	d_i	θ_i
1	0	90°	d_1	θ_1
2	a_2	0°	0	θ_2
3	a_3	0°	0	θ_3
4	0	90°	d_4	θ_4
5	0	-90°	d_5	θ_5
6	0	0°	d_6	θ_6

Tabelle 2.2: DH-Parameter für die Gelenke des Roboters [10]

Die Tabellle 2.2 Zeigt die Deavit-Hartenberg der UR3e Roboter und hat folgonde Parameter

- **Gelenk i :** Die Nummer des jeweiligen Gelenks des Roboters. Es gibt insgesamt 6 Gelenke, nummeriert vom Basisgelenk (1) bis zum Endeffektor (6).
- a_i (**Linklänge**): Der Abstand zwischen den Achsen z_{i-1} und z_i entlang der Achse x_i . a_i ist die kinematische Länge des i -ten Glieds.[11]
- α_i : Der Winkel, um den die Achse z_{i-1} um die Achse x_i rotiert werden muss, damit sie parallel zur Achse z_i wird. (Drehung um x_i)[11]
- d_i : Der Abstand zwischen den Achsen x_{i-1} und x_i entlang der Achse z_{i-1} . (Translation entlang z_{i-1})[11]
- θ_i : Der Winkel, um den die Achse x_{i-1} um die Achse z_{i-1} rotiert werden muss, damit sie parallel zur Achse x_i wird. (Drehung um z_{i-1})[11]

Mithilfe der Kenntnisse der Davit- Hartenberg Methode lässt sich die Position und die Orientierung des TCP berechnet. Für die Berechnungen werden sogenannte Transformationsmatrizen eingesetzt. Der TCP ist dabei entscheidend, um die genaue Position des Roboters im Raum zu

bestimmen. Mithilfe der Transformationsmatrizen lassen sich die Orientierung und Position der einzelnen Gelenke miteinander verknüpfen, was es ermöglicht, die Gesamtposition des Werkzeugkopfes im Raum präzise zu bestimmen.

Die Notation $R \vdash R \parallel R \parallel R \vdash R \vdash R \parallel R$ steht dabei exemplarisch für die relative Lage und die Ausrichtung der Gelenke zueinander. Diese Darstellung ist grundlegend für die Bildung der Gesamttransformationsmatrix.

In den folgenden Abschnitten werden die Transformationsmatrizen jedes einzelnen Gelenks beschrieben sowie die Berechnung der Gesamtmatrix T , die die Position und Orientierung des Endeffektors bestimmt.

$R \vdash R(90^\circ)$:

$${}^0T_1, {}^3T_4 = \begin{bmatrix} \cos(\theta_i) & 0 & \sin(\theta_i) & 0 \\ \sin(\theta_i) & 0 & -\cos(\theta_i) & 0 \\ 0 & 1 & 0 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R \vdash R(-90^\circ)$:

$${}^4T_5 = \begin{bmatrix} \cos(\theta_i) & 0 & -\sin(\theta_i) & 0 \\ \sin(\theta_i) & 0 & \cos(\theta_i) & 0 \\ 0 & 1 & 0 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$R \parallel R(0^\circ)$:

$${}^1T_2, {}^2T_3, {}^5T_6 = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i) & 0 & a_i \cdot \cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i) & 0 & a_i \cdot \sin(\theta_i) \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Nachdem die Transformationsmatrizen für jedes Gelenk festgelegt wurden, kann die Gesamttransformationsmatrix zur Bestimmung des TCP ermittelt werden. Dies geschieht durch die sukzessive Multiplikation der einzelnen Matrizen in der richtigen Reihenfolge

$$T_{Gesamt} = {}^0T_1 \cdot {}^1T_2 \cdot {}^2T_3 \cdot {}^3T_4 \cdot {}^4T_5 \cdot {}^5T_6 \quad (2.1)$$

Der Abstand zwischen zwei Gelenken wird durch die Variable L dargestellt. Um die Matrixdarstellung übersichtlicher zu gestalten, wird für den Verschiebungsvektor eine vereinfachte Notation verwendet.

$$T_{Gesamt} = \begin{bmatrix} \cos(\theta_1 + \theta_2 + \dots + \theta_6) & -\sin(\theta_1 + \theta_2 + \dots + \theta_6) & 0 & \sum_{i=1}^6 L_i \cdot \cos(\sum_{i=1}^n \theta_i) \\ \sin(\theta_1 + \theta_2 + \dots + \theta_6) & \cos(\theta_1 + \theta_2 + \dots + \theta_6) & 0 & \sum_{i=1}^6 L_i \cdot \sin(\sum_{i=1}^n \theta_i) \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Aus der Gesamttransformationsmatrix lassen sich der Verschiebungsvektor zur Bestimmung der TCP-Position sowie die Rotationsmatrix $\mathbf{R}$ zur Orientierung des Roboters ablesen. Für den Verschiebungsvektor zur Bestimmung der TCP-Position gilt:

$$TCP = \sum_{i=1}^n L_n \cos(\sum_{i=1}^n \theta_i) \cdot \vec{x} + \sum_{i=1}^n L_n \sin(\sum_{i=1}^n \theta_i) \cdot \vec{y} + 0 \cdot \vec{z}$$

$$Orientierung = \begin{bmatrix} \cos(\theta_1 + \theta_2 + \dots + \theta_6) & -\sin(\theta_1 + \theta_2 + \dots + \theta_6) & 0 \\ \sin(\theta_1 + \theta_2 + \dots + \theta_6) & \cos(\theta_1 + \theta_2 + \dots + \theta_6) & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Nach der Aufstellung der Formel für den TCP, folgt nun eine Beschreibung des Werkzeugkopfes und seiner Funktionalität.

Der in dieser Bachelorarbeit eingesetzte UR3e-Roboter ist mit dem Robotiq Hand-E Greifer ausgestattet, der speziell für den industriellen Einsatz konzipiert wurde. Der Greifer besitzt zwei parallel angeordnete Greiffinger, die durch einen einzelnen Motor bewegt werden. Durch dieses Design ist es möglich, sowohl äußere als auch innere Greifvorgänge durchzuführen. Dabei können die Finger hohle Werkstücke von innen besser greifen, indem sie Druck nach außen ausüben. Der Greifer erreicht dabei eine maximale Greifkraft von 185 N und bietet einen verstellbaren Greifweg von 0 bis 50 mm, wodurch er sich für unterschiedlich große Objekte eignet. Darüber hinaus überzeugt der Hand-E Greifer durch seine hohe Genauigkeit und einfache Handhabung, was ihn ideal für präzise Montageaufgaben macht[12].

Es ist jedoch wichtig zu beachten, dass gemäß dem Handbuch des Herstellers nach dem Aufnehmen der normalen Nutzlast der Roboter mit dem Hand-E Greifer eine Kraft von bis zu 100 N in jede Richtung ausüben kann. Ein Überschreiten dieser Grenze kann den Greifer beschädigen oder zum Verlust der Nutzlast führen[13].



Abbildung 2.5: Robotiq Hand-E Greifer [13]

Der Robotiq Hand-E Greifer funktioniert mit einem Gleichstrommotor. Dieser wird unter einer Spannung von $24V \pm 10\%$ versorgt, wobei Stromstöße von bis zu 1,1A erwartet werden können. Die Kommunikation des Greifers erfolgt über das serielle Kommunikationsprotokoll Modbus RTU RS485 (Remote Terminal Unit). Das Pinout umfasst Anschlüsse für Versorgungsspannung (24V und GND) sowie die Datenleitungen (A und B) für die serielle Kommunikation[13].

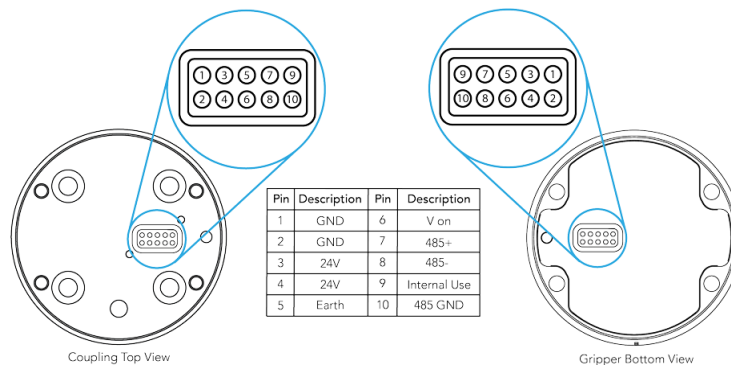


Abbildung 2.6: Die Verbindung zwischen dem Greifer und seiner Kupplung[13]

Wie bereits erwähnt, besteht die zweite Aufgabe dieser Bachelorarbeit darin, die Position der Nadelspitze sowie des Fadens mithilfe einer Kamera zu erfassen und die entsprechenden Koordinaten im globalen Koordinatensystem an den zweiten Roboterarm zu übermitteln. Die Erfassung der Nadelspitze erfolgt durch Bildverarbeitung mit einer uEye-Kamera, deren Funktionsweise im folgenden Abschnitt näher erläutert wird.

2.2 Vorstellung des uEye Kamera

Die IDS uEye CP Rev. 2 ist eine kompakte Industriekamera, die sich durch ihre hohe Bildqualität und eine robuste Bauweise auszeichnet. Sie ist für eine Vielzahl industrieller Bildverarbeitungsanwendungen konzipiert und bietet je nach Modell CMOS-Sensoren mit Auflösungen von bis zu 20 Megapixel. Dabei sind sowohl verschiedene Farb- als auch Monochromvarianten möglich.

Die Kamera unterstützt hohe Bildraten von bis zu 160 Bildern pro Sekunde (abhängig von Auflösung und Sensortyp) und ermöglicht durch ihre Gigabit-Ethernet- und USB3-Vision-Schnittstelle eine schnelle und zuverlässige Datenübertragung. Typische Funktionen wie Hardware-Triggerung, Automatische Belichtungsregelungen, die Verstärkungssteuerung (Gain), sowie ROI-Einstellungen (Region of Interest) sind integriert und über die IDS-Software steuerbar.

Dank ihrer kompakten Bauform mit 29 x 29 x 49.1 mm und einem robusten Aluminiumgehäuse eignet sich die Kamera besonders für Anwendungen in begrenztem Raum. Typische Einsatzgebiete sind unter anderem die industrielle Qualitätskontrolle, Robotik, Mikroskopie oder Verkehrstechnik.

Für die Softwareanbindung steht das IDS Software Development Kit (uEye SDK) zur Verfügung, welches eine einfache Integration unter Windows und Linux erlaubt. Modelle mit Farbsensor ermöglichen zusätzlich die Erfassung von RGB-Bildern, was insbesondere für Aufgaben in der Farb-erkennung oder Objektklassifikation von Vorteil ist.[24]

Aufgrund dieser technischen Eigenschaften stellt die uEye CP Rev. 2 eine leistungsstarke und vielseitige Lösung für die industrielle Bildverarbeitung dar.

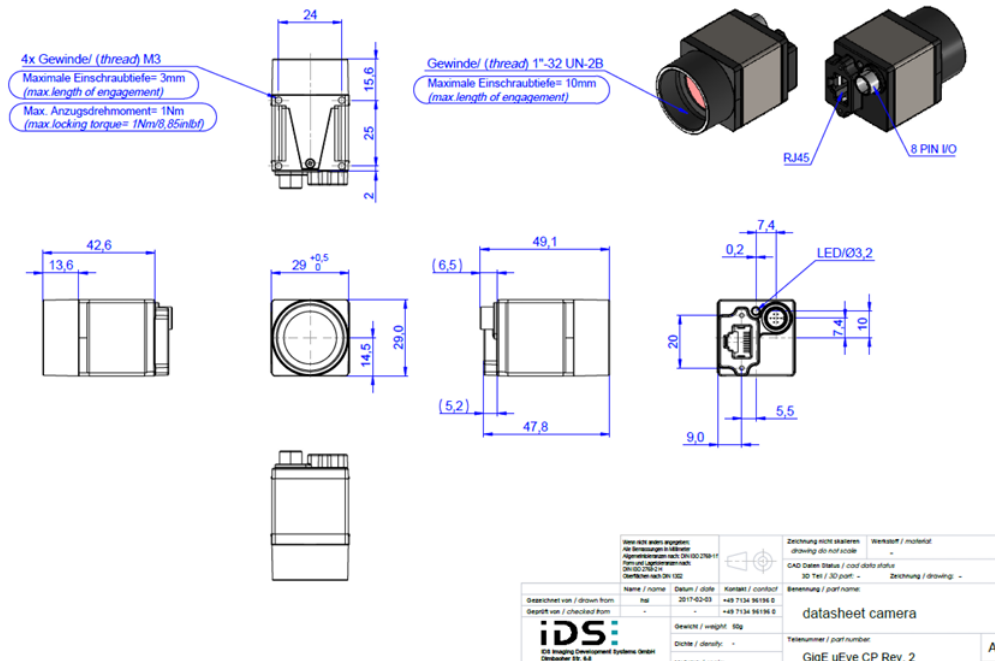


Abbildung 2.7: Mechanische Spezifikationen der uEye Kamera [24]

• RJ45-Netzwerkanschluss (GigE)

Die uEye CP Rev. 2 Kamera ist mit einem **RJ45-Anschluss** ausgestattet, über den die Datenübertragung mittels *Gigabit Ethernet (GigE Vision)* erfolgt. Dies ermöglicht eine zuverlässige und leistungsfähige Verbindung zwischen Kamera und Host-PC und erlaubt dabei die Übertragung großer Bilddatenmengen mit hoher Geschwindigkeit und geringer Latenz. Zudem unterstützt die GigE-Verbindung die *Stromversorgung über Ethernet (PoE)*, wodurch eine separate Spannungsversorgung der Kamera entfallen kann, sofern der Host diese Funktion unterstützt. Der RJ45-Anschluss entspricht dem Industriestandard und eignet sich somit gut für den dauerhaften Einsatz in industriellen Umgebungen.

• 8-Pin I/O-Anschluss

Zusätzlich verfügt die Kamera über einen **8-poligen I/O-Anschluss** (8-Pin Hirose-Buchse), der zur Ansteuerung externer Komponenten dient. Darüber können digitale Signale ein- und ausgegeben werden, beispielsweise zur Anbindung von Triggergebern, Beleuchtungseinheiten oder anderen Aktoren. Die I/O-Schnittstelle unterstützt folgende Funktionen:

- Digitaler Eingang (z. B. für externe Triggerung),
- Digitaler Ausgang (z. B. zur Synchronisation externer Geräte),
- Stromversorgung externer Komponenten (je nach Kameraausstattung und Pin-Belegung).

Die genaue Belegung und Signaldefinition der 8-Pin-I/O-Schnittstelle ist dem jeweiligen Datenblatt oder der technischen Dokumentation des IDS zu entnehmen.

2.3 Technische Grundlagen von Visual Servoing und andere Methode

Visuelles Servoing (auch als visuelles Sehen bezeichnet) ist eine Methode zur Echtzeitsteuerung der Bewegung von Robotern mithilfe von Kameradaten. Dabei werden visuelle Informationen

aus der Umgebung oder von einem Zielobjekt präzise analysiert, um daraus geeignete Steuerbefehle für den Roboter abzuleiten[14]. In der Literatur werden verschiedene Methoden des Visual Servoing beschrieben. Zwei grundlegende Ansätze, die Gegenstand dieser Arbeit sind, sind das *Position-based Visual Servoing (PBVS)* und das *Image-based Visual Servoing (IBVS)*.

In diesem Abschnitt werden grundsätzlich zwei Echtzeit-Ansätze der bildbasierten Regelung unterschieden: PBVS und IBVS. Beide Ansätze basieren auf dem Einsatz von einem oder mehreren Kamera. Die spezifische Konfiguration spielt dabei für die Auswahl der jeweiligen Steuerungsmethode keine entscheidende Rolle.

Es gibt darüber hinaus weitere Ansatz zur Implementierung hybrider Verfahren, die PBVS und IBVS kombinieren, um die Vorteile beider Methoden in einem System zu vereinen. Auf diese hybriden Ansätze wird in dieser Arbeit jedoch nicht weiter eingegangen, da sie nicht wichtig für die hier verfolgten Ziele sind.

Beide Steuerungsmethoden, *PBVS* und *IBVS*, sind für ihre spezifische Konzeption und jeweiligen Anwendungsbereiche im Visual Servoing sehr effektiv. Sie unterscheiden sich jedoch in ihrer Funktionsweise und weisen jeweils spezifische Vor- und Nachteile auf. Daher ist es zunächst notwendig, die grundlegenden Mechanismen beider Ansätze klar voneinander abzugrenzen.

Das PBVS nutzt Echtzeitbildgebung in Kombination mit einem berechneten 3D-Modell. Dabei wird ein Bildfehler s bzw. ein Soll-Bildfehler s^* ermittelt, indem beide Informationsquellen das 3D-Modell und das aktuelle Kamerabild miteinander verglichen und ausgewertet werden.

Bei der PBVS-Methode wird der Soll-Bildfehler s^* verwendet, um die Position und Orientierung des Roboters im dreidimensionalen kartesischen Raum zu schätzen. Das bedeutet, dass die Ausrichtung und Führung des Endeffektors algorithmisch ausschließlich auf Grundlage visueller Merkmale bestimmt wird, ohne feste Referenzpunkte wie Kanten, Ecken oder andere markante Bildmerkmale zu nutzen.

Die PBVS-Methode arbeitet auf Basis eines Korrelationsmodells: Aus dem Vergleich von Soll- und Ist-Bild wird ein Fehler berechnet, auf dessen Grundlage Korrekturen an Position und Orientierung des Roboters vorgenommen werden. Dadurch nähert sich der Roboter iterativ der gewünschten Zielposition an.

Das kartesische Steuerungsgesetz von PBVS erlaubt es, sowohl lineare Geschwindigkeiten V als auch Winkelgeschwindigkeiten ω direkt zu beeinflussen, sodass translatorische Bewegungen in (x^*, y^*, z^*) und rotatorische Bewegungen in $(\alpha^*, \beta^*, \lambda^*)$ auf Basis des Bildfehlers berechnet werden können.[15]

Die praktische Umsetzung der PBVS-Methode wird jedoch durch zwei wesentliche Anforderungen eingeschränkt:

- Es müssen detaillierte Informationen über das Erscheinungsbild und die Geometrie des Zielobjekts vorliegen.
- Es ist eine äußerst präzise Kalibrierung des integrierten Kamerasystems erforderlich.

Die Schwierigkeit dieser Methode besteht darin, dass eine 3D-Rekonstruktion schwer durchzuführen ist. Ihre Architektur gestaltet sich wie folgt:

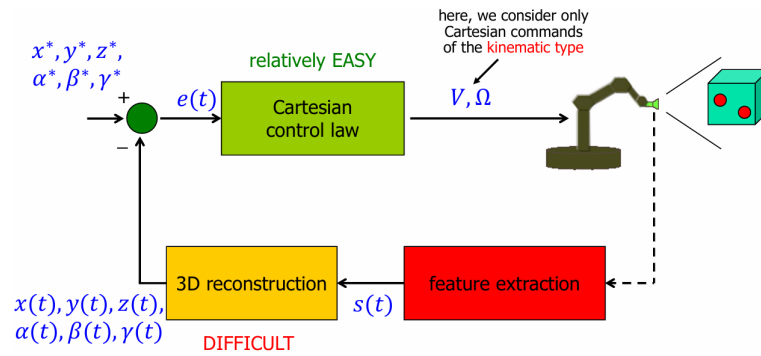


Abbildung 2.8: PBVS Architecture[15]

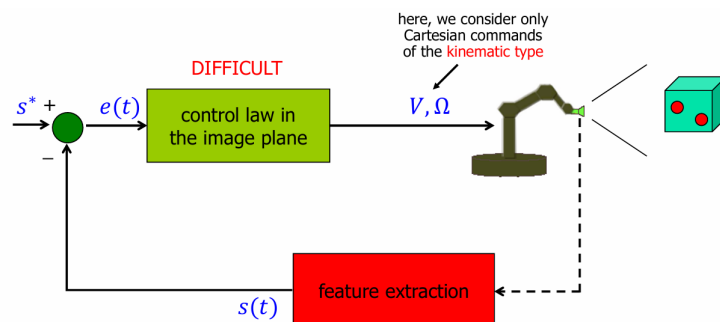
Die Methode des IBVS verfolgt eine grundlegend andere Herangehensweise als PBVS. Während PBVS auf der Schätzung der dreidimensionalen Position des Zielobjekts basiert, nutzt IBVS fest definierte Bildmerkmale direkt in der zweidimensionalen Bildebene, um den Soll-Ist-Bildfehler zu bestimmen.

Als solche Bildmerkmale können Kanten, Markierungen oder andere charakteristische Strukturen dienen; eine explizite Berechnung der Position des Zielobjekts im Raum ist nicht erforderlich. Stattdessen werden die erfassten Bilddaten einer zweidimensionalen Kamera von einer Recheneinheit interpretiert und genutzt, um diese Merkmale als Referenz- bzw. Orientierungspunkte für die Regelung heranzuziehen.

Dies bedeutet, dass die kameragestützte Recheneinheit des Systems bei IBVS anders als bei PBVS keine dreidimensionalen Informationen verarbeitet, sondern ausschließlich auf zweidimensionale Bilddaten zugreift.

IBVS nutzt zur Berechnung des Bildfehlers s die zuvor erfassten Bildinformationen und spezifische Bildmerkmale zum Zeitpunkt $t - 1$ sowie das Soll-Bild, auch Referenzbild genannt. Dabei werden die Bildinformationen des aktuellen Zustandsbildes und des Referenzbildes auf Pixelebene miteinander verglichen, um den Bildfehler s zu bestimmen, um diesen anschließend zu minimieren. Auf Basis dieses Bildfehlers generiert das System Steuerbefehle, die an die Aktuatoren übermittelt werden, um die Bewegung des Roboters entsprechend zu korrigieren.

Die architecture sieht dann wie folgt aus.

Abbildung 2.9: IBVS Architecture [15]
Visual Servicing

Wir können anschließend die folgende Vergleichstabelle zwischen PBVS und IBVS aufstellen, die zugleich als Übersicht der jeweiligen Vor- und Nachteile interpretiert werden kann.

Kriterium	PBVS	IBVS
Steuerungsgröße	3D Pose	2D Bildkoordinaten
Modellabhängigkeit	Benötigt Modell	Kein Modell nötig
Robustheit	Störungsrobust (aber kalibrierungsabhängig)	Sensitiv bei Feature-Verlust
Einfachheit	Aufwändig (Pose-Schätzung)	Einfacher Ansatz

Tabelle 2.3: Vergleich zwischen PBVS und IBVS

Wegen des geringeren technischen Aufwandes, wird in dieser Arbeit die IBVS-Methode verwendet.

Kapitel 3

Methodik

In diesem Abschnitt werden der Ablauf der Entwicklung, die zugrunde liegenden mathematischen Grundlagen und die eingesetzten Softwarekomponenten erläutert.

3.1 Entwurf und Konzeption der kontinuierlichen Bewegung der Nadelspitze innerhalb der Wunde

Wie in der Beschreibung des Ist-Zustands bereits erläutert, ist die Bewegung der Nadelspitze innerhalb der Wunde nicht wie erwünscht. Die Bewegung erfolgt in mehreren Einzelschritten und ist dadurch diskontinuierlich. Ein Ziel dieser Arbeit ist es, die Bewegung der Nadelspitze so zu optimieren, dass diese kontinuierlich und gleichmäßig erfolgt, um Gewebeschädigungen zu minimieren und die Qualität der Nahtführung zu verbessern.

Daraus ergibt sich die grundlegende Frage, wann eine Trajektorie als kontinuierlich gilt. Aus mathematischer Sicht gilt eine Trajektorie dann als kontinuierlich, wenn ihre erste Ableitung bezüglich des Wegparameters existiert. Mit anderen Worten: Eine Funktion, die differenzierbar ist, ist zugleich stetig, da Differenzierbarkeit stets Stetigkeit impliziert.

Eine der mathematischen Funktionen, die in diesem Zusammenhang unsere Aufmerksamkeit auf sich gezogen hat, ist die elliptische Funktion. Grund dafür ist die Geometrie der Nadelbewegung, die in ihrer Form einer Ellipse ähnelt. Die entsprechenden charakteristischen Gleichungen für die beiden Arten der Ellipse, nämlich die große Halbachse und die kleine Halbachse, lauten wie folgt:

$$\frac{(x-0)^2}{a^2} + \frac{(y-0)^2}{b^2} = 1, a > b$$

oder

$$\frac{(x-0)^2}{b^2} + \frac{(y-0)^2}{a^2} = 1, b > a [16]$$

die entsprechenden ersten Ableitungen von den Formeln sehen wie folgt aus:

$$\frac{dy}{dx} = -\frac{b^2}{a^2} \cdot \frac{x}{y}$$

oder

$$\frac{dy}{dx} = -\frac{a^2}{b^2} \cdot \frac{x}{y} [17]$$

Was immer existiert. Diese elliptischen Funktionen sind dann kontinuierlich. Die entsprechenden Kurven der oben genannten Formeln sehen wie folgt aus:

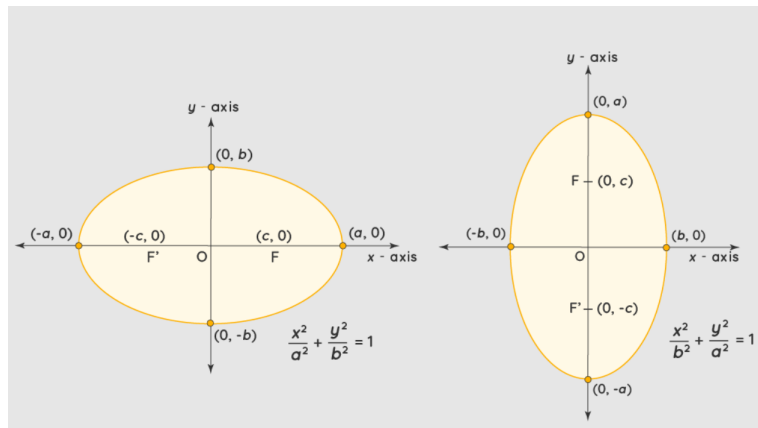


Abbildung 3.1: Elliptische Kurven [16]

Anhand dieser elliptischen Funktionen soll die Nadelspitze präzise in die Wunde. Auf diese Weise lässt sich eine kontinuierliche Trajektorie realisieren.

Dabei muss außerdem definiert werden, welche Größe die Ellipse einnehmen darf. Ein zentraler Parameter ist hierbei der Abstand des Ellipsenpunkts vom Ursprung, der sogenannte Radius, der in Abhängigkeit der Halbachsen a , b und des Winkels ϕ ausgedrückt werden kann. Um diesen Zusammenhang herzuleiten, folgt nun eine Betrachtung der Ellipse in einem geeigneten Koordinatensystem $(O, \vec{Y}, \vec{Z}, \vec{X})$.

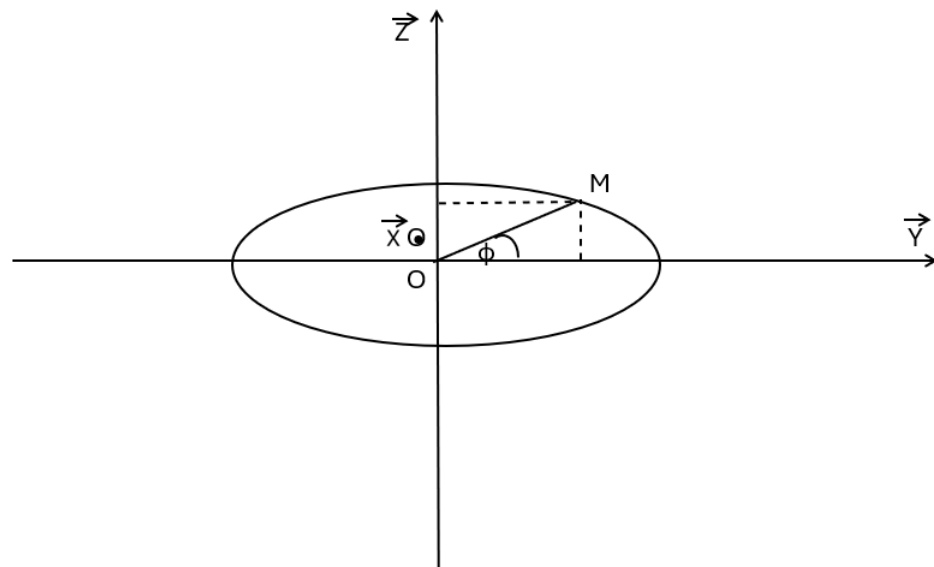


Abbildung 3.2: Ellipse in ein Koordinatensystem

Die charakteristische Gleichung für die obere Ellipse ist :

$$\frac{y^2}{a^2} + \frac{z^2}{b^2} = 1$$

Der Radius der Ellipse wird beschrieben durch:

$$r^2 = |\vec{OM}|^2 = y^2 + z^2 \quad (3.1)$$

Formel 3.2 nach z^2 aufgelöst:

$$\begin{aligned} \frac{y^2}{a^2} + \frac{z^2}{b^2} &= 1 \\ \Rightarrow z^2 &= b^2 \left(1 - \frac{y^2}{a^2}\right) \end{aligned} \quad (3.2)$$

Formel 3.2 eingesetzt in Formel 3.1:

$$\begin{aligned} r^2 &= y^2 + z^2 \\ r^2 &= y^2 + b^2 \left(1 - \frac{y^2}{a^2}\right) \\ r^2 &= b^2 \left(1 - \frac{y^2}{a^2}\right) + y^2 \\ r^2 &= b^2 - b^2 \cdot \frac{y^2}{a^2} + y^2 \\ r^2 &= b^2 - (b^2 - a^2) \cdot \frac{y^2}{a^2} \end{aligned} \quad (3.3)$$

$$y = r \cdot \cos(\phi) \quad (3.4)$$

$$z = r \cdot \sin(\phi) \quad (3.5)$$

Formel 3.4 eingesetzt in Formel 3.3 :

$$\begin{aligned} r^2 &= b^2 - (b^2 - a^2) \cdot \frac{r \cdot \cos(\phi)^2}{a^2} \\ r^2 &= b^2 - (b^2 - a^2) \cdot \frac{r^2 \cdot \cos(\phi)^2}{a^2} \end{aligned} \quad (3.6)$$

Formel 3.6 nach r^2 aufgelöst:

$$\begin{aligned} r^2 &= b^2 - (b^2 - a^2) \cdot \frac{r^2 \cdot \cos(\phi)^2}{a^2} \\ r^2 - b^2 &= (-b^2 + a^2) \cdot \frac{r^2 \cdot \cos(\phi)^2}{a^2} \\ r^2 &= \frac{a^2 \cdot b^2}{a^2 + \cos(\phi)^2 \cdot (b^2 - a^2)} \\ r^2 &= \frac{a^2 \cdot b^2}{a^2 \cdot \sin^2(\phi) + b^2 \cdot \cos^2(\phi)} \end{aligned} \quad (3.7)$$

Damit erhalten wir die Formel für den Radius in Abhängigkeit von den Parametern a , b und ϕ :

$$r = \frac{a \cdot b}{\sqrt{b^2 \cdot \cos^2(\phi) + a^2 \cdot \sin^2(\phi)}}$$

Der Positionsvektor der Nadelspitze auf der Ellipse ist dann

$$\begin{aligned} P(\varphi) &= 0 \cdot \hat{x} + y \cdot \hat{y} + z \cdot \hat{z} \\ P(\varphi) &= 0 \cdot \hat{x} + r \cdot \cos(\varphi) \cdot \hat{y} + r \cdot \sin(\varphi) \cdot \hat{z} \end{aligned} \quad (3.8)$$

Die Ableitung des Positionsvektors nach φ Dadurch erhält man die Tangente an dem Punkt, welcher der Ausrichtung der Nadelspitze entspricht .

$$\begin{aligned} \frac{\delta P(\varphi)}{\delta \varphi} &= \vec{O}(\varphi) = \dot{r} \cdot \cos(\varphi) \cdot \hat{y} - r \cdot \sin(\varphi) \cdot \hat{y} + \dot{r} \cdot \sin(\varphi) \cdot \hat{z} + r \cdot \cos(\varphi) \cdot \hat{z} \\ &= \hat{y}(\dot{r} \cdot \cos(\varphi) - r \cdot \sin(\varphi)) + \hat{z}(\dot{r} \cdot \sin(\varphi) + r \cdot \cos(\varphi)) \end{aligned} \quad (3.9)$$

Berechnung von $\dot{r}$:

$$\begin{aligned} \frac{\delta r}{\delta \varphi} &= \frac{\delta(r^2)^{\frac{1}{2}}}{\delta \varphi} \\ &= \frac{\delta}{\delta \varphi} \cdot \left(\frac{a^2 \cdot b^2}{a^2 + \cos(\varphi)^2 \cdot (b^2 - a^2)} \right)^{\frac{1}{2}} \\ &= a \cdot b \cdot \frac{\delta}{\delta \varphi} \cdot \left((a^2 + \cos(\varphi)^2 \cdot (b^2 - a^2)) \right)^{-\frac{1}{2}} \\ &= \frac{1}{2} \cdot a \cdot b \cdot (a^2 + (b^2 - a^2) \cdot \cos(\varphi)^2)^{-\frac{3}{2}} \cdot (b^2 - a^2) \cdot 2 \cdot \sin(\varphi) \cdot \cos(\varphi) \\ \dot{r} &= \frac{a \cdot b \cdot (b^2 - a^2) \cdot \sin(\varphi) \cos(\varphi)}{(a^2 + (b^2 - a^2) \cdot \cos(\varphi)^2)^{\frac{3}{2}}} \end{aligned}$$

Ein weiteres Problem besteht darin, dass die Steuerung nicht direkt an der Nadelspitze erfolgt, sondern über den TCP des Roboters. Das nachfolgende Bild veranschaulicht , wie der TCP in relation zur Nadelspitze positioniert ist.

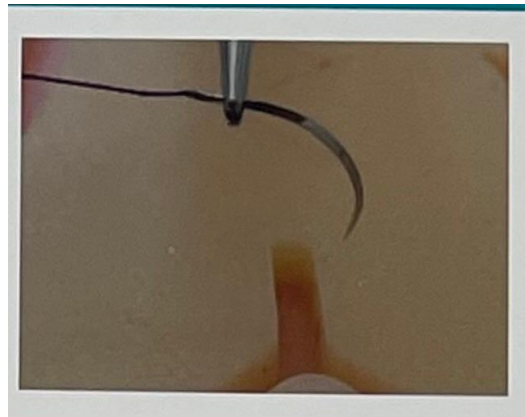


Abbildung 3.3: TCP des Roboters und Nadelspitze

Es ergibt sich folgende Frage: Wie muss der Roboter am TCP gesteuert werden, damit die Nadelspitze der gewünschten elliptischen Bahn folgt? Diese Frage ist folgend mithilfe eines Blockdiagramm dargestellt.

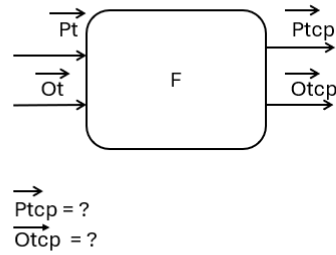


Abbildung 3.4: Blocksystem

Dazu muss die entsprechenden Zusammenhang zwischen den $\vec{P}_{tcp}$ und $\vec{P}_t$ gefunden werden. Um diese Zusammenhang zu finden, wird im folgenden die Position des TCP und die der Nadelspitze in einem globalen Koordinatensystem veranschaulicht.

Aus den Blocksystem können wir den Gleichung aufstellen $\vec{p}_t = F(\vec{P}_{tcp})$

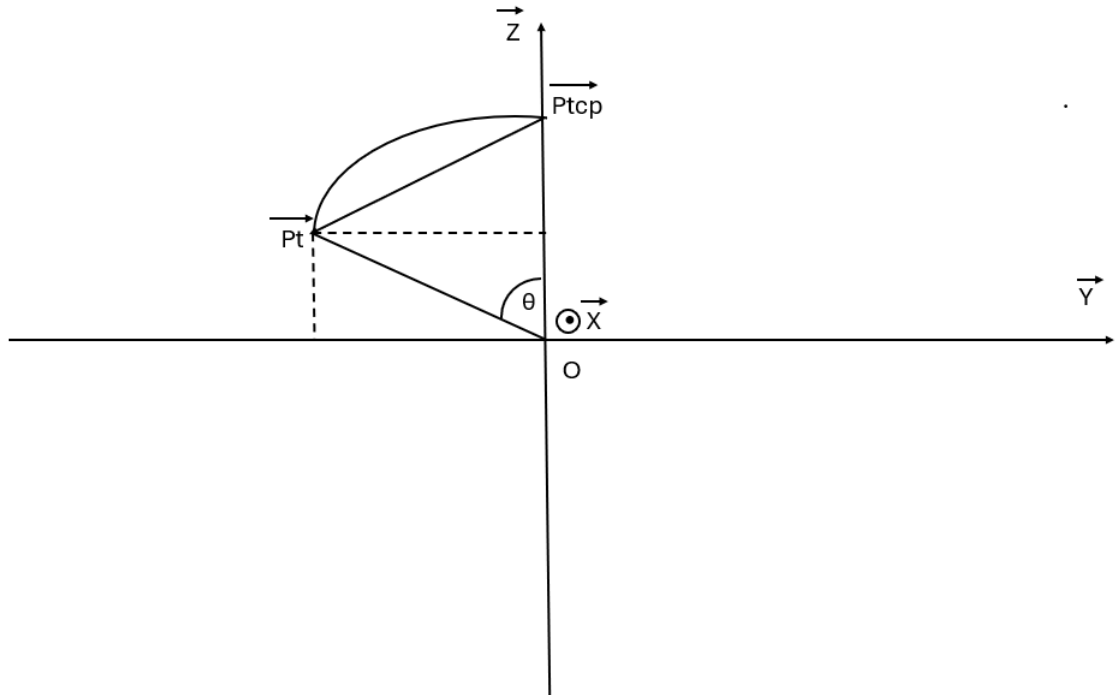


Abbildung 3.5: Position des TCP und der Nadelspitze im Global Koordinatensystem

Daraus lässt sich folgende Gleichung herleiten:

Für $\vec{P}_{tcp}$ gilt

$$\vec{P}_{tcp} = \begin{pmatrix} 0 \\ 0 \\ r_n \end{pmatrix}$$

für $\vec{P}_t$ gilt

$$\vec{P}_t = \begin{pmatrix} 0 \\ -r_n \sin(\theta) \\ r_n \cos(\theta) \end{pmatrix}$$

Festzustellen ist, dass sich die Position der Nadelspitze $\vec{P}_t$ stets aus der Position des TCP $\vec{P}_{tcp}$ und einem Verschiebungsvektor $\vec{\Delta}_p$ ergibt, der vom TCP zur Nadelspitze zeigt. Mathematisch lässt sich dieser Zusammenhang durch die folgende Gleichung ausdrücken:

$$\vec{P}_t = \vec{P}_{tcp} + \vec{\Delta}_p$$

mit:

$$\vec{\Delta}_p = \vec{P}_t - \vec{P}_{tcp} = \begin{pmatrix} 0 \\ -r_n \sin(\theta) \\ r_n (\cos(\theta) - 1) \end{pmatrix}$$

Um die oben dargestellte Gleichung zu verifizieren und deren Verhalten zu veranschaulichen, wird dies im folgenden durch ein entsprechendes Python-Skript veranschaulicht. Dabei wird das Verhalten des Vektors bei verschiedenen Winkeln θ simuliert. Die Ergebnisse dieser Simulation sind nachfolgend dargestellt.

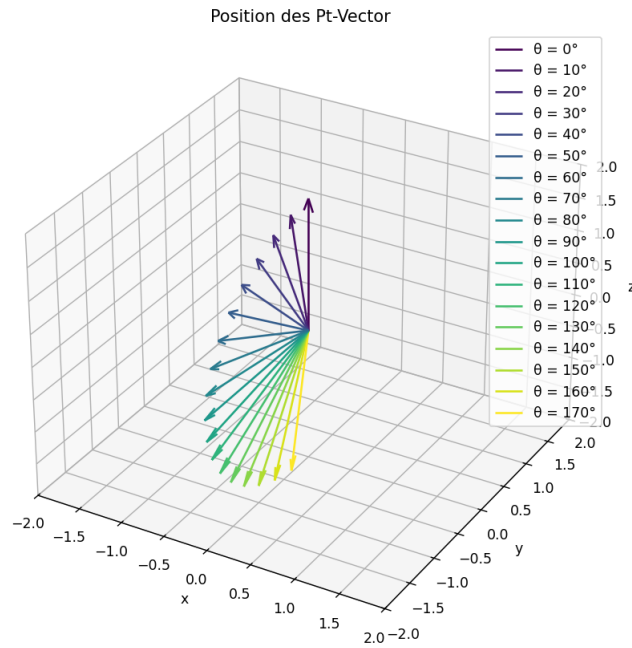


Abbildung 3.6: Formelsimulation des Vektors vom TCP zur Nadelspitze

Aus der zuvor hergeleiteten Beziehung zwischen $\vec{P}_t$ und $\vec{P}_{tcp}$ folgt, dass sich die Position des TCP direkt aus der bekannten Nadelspitzenposition $\vec{P}_t$ berechnen lässt. Da $\vec{P}_t$ immer auf der vorgegebenen Ellipsenbahn liegt, genügt dazu eine einfache Rücktransformation der gefundenen Gleichung:

$$\vec{P}_{tcp} = \vec{P}_t - \begin{pmatrix} 0 \\ -r_n \sin(\theta) \\ r_n (\cos(\theta) - 1) \end{pmatrix}$$

Für die Orientierung des TCP O_{tcp} wurde die berechnete Tangentenrichtung der elliptischen Bahn übernommen, sodass sich der Endeffektor stets entlang der Bewegungsrichtung ausrichtet:

$$\vec{O}_{tcp} = \frac{\delta P(\varphi)}{\delta \varphi} = \hat{y}(\dot{r} \cdot \cos(\varphi) - r \cdot \sin(\varphi)) + \hat{z}(\dot{r} \cdot \sin(\varphi) + r \cdot \cos(\varphi))$$

Bevor die hergeleitete Formel in das Steuerungssystem des Roboters integriert wurde, erfolgt zunächst eine Simulation in Python. Ziel dieser Voruntersuchung ist es, das Verhalten der Formel zu analysieren und ihre Nachvollziehbarkeit sicherzustellen. Das Ergebnis dieser Simulation wird im Folgenden dargestellt.

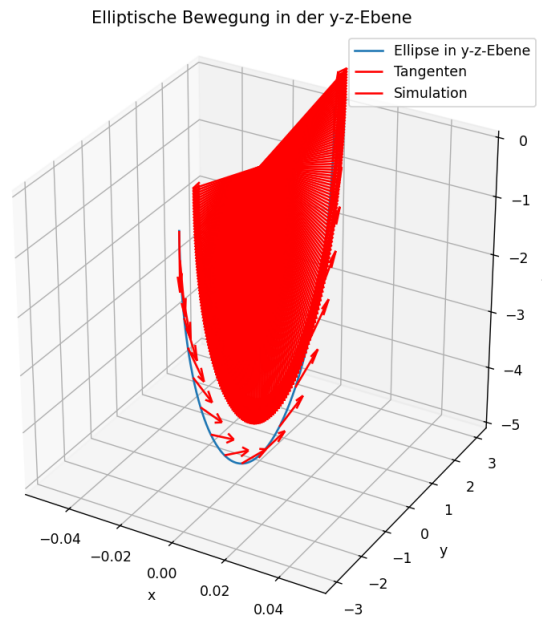


Abbildung 3.7: Formelsimulation der Bewegung

3.2 Entwurf und Umsetzung der Bildverarbeitung zur Erkennung von Nadel und Faden

Dieser Abschnitt beschäftigt sich mit der Bildverarbeitung. Ziel ist es, die Nadelspitze, sowie den Faden mithilfe der Kamera so zu erkennen, dass ihre Position im globalen Koordinatensystem präzise bestimmt werden kann. Diese Genauigkeit ist erforderlich, damit der Roboterarm in der Lage ist, gezielt und zuverlässig auf die Nadel und den Faden zuzugreifen. Wie bereits in einem vorherigen Abschnitt erläutert, wurde für die Umsetzung der Regelung die Methode der Image-Based Visual Servoing (IBVS) gewählt.

3.2.1 Vorgehensweise

Wie bereits im Abschnitt über die technischen Grundlagen des Visual Servoing erläutert, basiert das Image-Based Visual Servoing (IBVS) auf einem geschlossenen Regelkreis, bei dem der Roboter anhand des Bildfehlers gesteuert wird. In unserem Anwendungsfall lässt sich die Erkennung

von Nadel und Faden mithilfe mithilfe des IBVS-Modells anschaulich durch die folgende Abbildung darstellen:

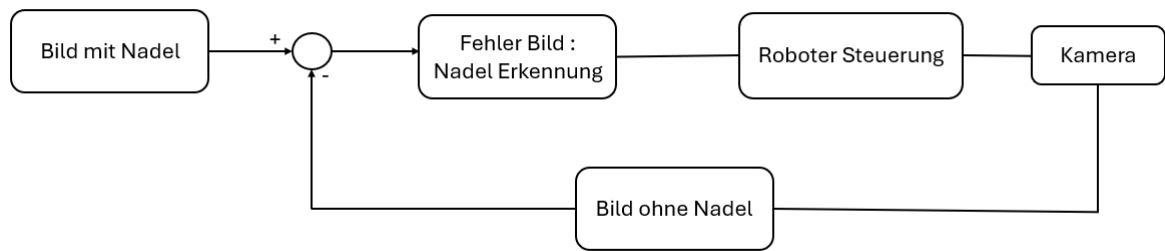


Abbildung 3.8: Erkennung der Nadel mithilfe des IBVS-Modells

Die obere Abbildung veranschaulicht den Ablauf der Nadelerkennung. Zunächst wird ein Bild aufgenommen, bevor die Nadel in den Erfassungsbereich eintritt. Anschließend wird ein zweites Bild aufgenommen, sobald die Nadel sichtbar ist. Durch Berechnung der Differenz beider Bilder entsteht ein Differenzbild, in dem alle unveränderten Bereiche auf einen Pixelwert von 0 (schwarz) gesetzt werden, während die Position der Nadel durch Pixelwerte von 255 (weiß) hervorgehoben wird. Anhand dieses Differenzbilds kann die Nadel zuverlässig detektiert werden. Der gleiche Prozess wird auch zur Erkennung des Fadens verwendet. Der gesamte Ablauf ist auch als Regelkreis organisiert und in der nachfolgenden Abbildung schematisch dargestellt.

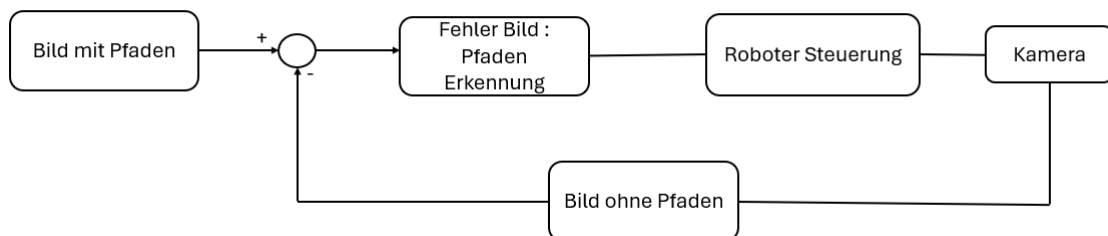


Abbildung 3.9: Erkennung des Fadens mithilfe des IBVS-Modells

Damit der Roboter die Nadel und den Faden präzise anfahren und sicher greifen kann, ist eine exakte Transformation vom Kamerakoordinatensystem in das globale Koordinatensystem erforderlich, da die Steuerung des Roboters ausschließlich in diesem globalen Koordinatensystem erfolgt. Folgend wird das Lochkameramodell erläutert, um zu veranschaulichen, wie die Kamera beim Aufnehmen eines Bildes arbeitet und wie ein Objekt im Bild dargestellt wird.

3.2.2 Lochkameramodell

Das Lochkameramodell (pinhole camera model) ist das einfachste Kameramodell, welches die mathematische Beziehung von Punkten im 3D-Raum auf einer Bildebene beschreibt [18]. Das Modell beschreibt die Projektion eines Punkt $P = (X, Y, Z)$ in einer zweidimensionalen Bildebene (u, v) . Dabei lässt sich das Prinzip in beide Richtungen anwenden. Es geht also um die

Abbildung eines dreidimensionalen Punktes auf eine einer zweidimensionalen Ebene.

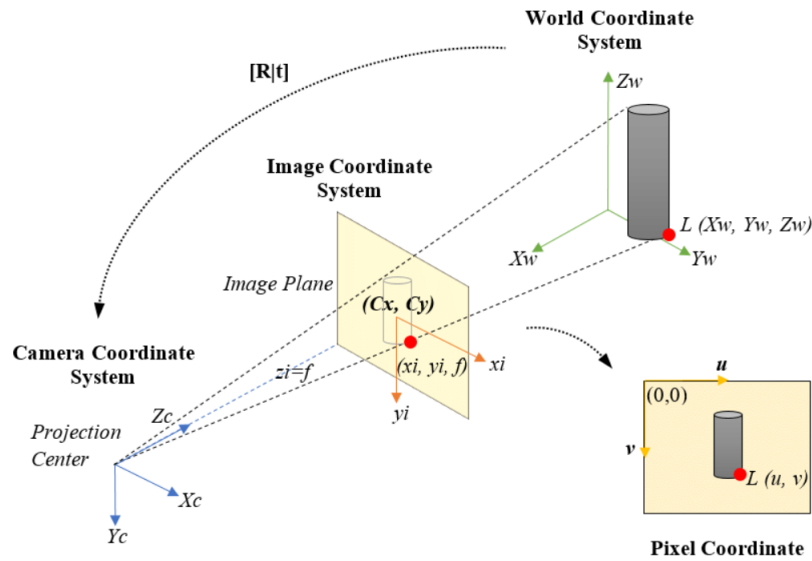


Abbildung 3.10: Lochkameramodell [19]

Die Koordinaten eines Punktes auf der Bildebene entsprechen den Koordinaten des farblich markierten Pixels im aufgenommenen Bild. Da es sich bei der Bildebene um eine zweidimensionale Fläche handelt, wird ein Bildpunkt P durch die Koordinaten u und v beschrieben. Diese Ebene befindet sich jedoch nicht direkt im Ursprung des Kamerakoordinatensystems, sondern hat einen definierten Abstand dazu. Dieser entspricht der Brennweite f der Kamera. Basierend auf diesen Informationen und unter Anwendung der Theorie der Strahlensätze[20] ist es möglich, Rückschlüsse auf die Position von Punkten im dreidimensionalen Raum zu ziehen.

Es gilt dann :

$$u = \frac{X}{Z} \cdot f + c_x, v = \frac{Y}{Z} \cdot f + c_y$$

Hierbei wird der Raumpunkt P durch die Kamera mit der Brennweite f erfasst. Die Brennweite f beschreibt den Abstand zwischen der Bildebene und dem Brennpunkt. Der Ursprung des Weltkoordinatensystems wird dabei auf die Bildebene projiziert. Die Werte c_x und c_y geben die Koordinaten des optischen Mittelpunkts (Principal Point) in der Bildebene an. Alle Parameter f , c_x und c_y werden im Rahmen der Kamerakalibrierung bestimmt.

3.2.3 Kalibrierung des uEye Kamera

Die Kalibrierung der Kamera ist ein essenzieller Schritt für die Bildverarbeitung, insbesondere wenn es darum geht, präzise Informationen aus Bilddaten zu extrahieren. Das Hauptziel der Kalibrierung besteht darin, die **intrinsischen** und **extrinsischen** Kameraparameter zu bestimmen.[21]

Zu den **intrinsischen Parametern** zählen unter anderem die **Kameramatrix** (K), welche die internen optischen Eigenschaften wie Brennweite und Hauptpunkt beschreibt, der **Verrzerrungskoeffizient** (dist), welcher die Abbildungsfehler der Linsenverzerrung modelliert

Die **extrinsischen Parameter** bestehen aus dem **Rotationsvektor** $\vec{r}$ und dem **Translationsvektor** $\vec{t}$, welche die Lage und Orientierung der Kamera relativ zu einem externen Weltkoordinatensystem beschreibt.[22]

Mithilfe der **Kameramatrix** lässt sich der Übergang von Koordinaten in der Bildebene in das **Kamerakoordinatensystem** durchführen. Die Kameramatrix basiert auf dem sogenannten *Lochkamera-modell*, aus dem sich folgende Funktion mathematisch herleiten lässt:

$$u = \frac{x_k}{z_k} \cdot f + c_x, v = \frac{y_k}{z_k} \cdot f + c_y$$

Mit einer formal eingeführten Größe d kann man dies formel wie folgt umschreiben:

$$\begin{aligned} d \cdot u &= f \cdot x_k + c_x \cdot z_k \\ d \cdot v &= f \cdot y_k + c_y \cdot z_k \\ d &= z_k \end{aligned}$$

Das Ganze lässt sich in Matrixform wie folgt darstellen:

$$\begin{pmatrix} d \cdot u \\ d \cdot v \\ d \end{pmatrix} = \begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} f & 0 & c_x & 0 \\ 0 & f & c_y & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \cdot \begin{pmatrix} x_k \\ y_k \\ z_k \\ 1 \end{pmatrix} = \tilde{\mathbf{K}} \cdot \tilde{\mathbf{x}}_K = \mathbf{K} \cdot \mathbf{x}_K [22]$$

Daraus ergibt sich dann die Kamera Matrix

$$K = \begin{pmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{pmatrix}$$

Durch die Verwendung des **Verzerrungskoeffizienten** ist es möglich, die vom Kamerasystem aufgenommenen Bilder zu entzerren. Beim Erfassen eines Bildes treten durch die optischen Eigenschaften der Kameralinse Verzerrungen auf, insbesondere radiale und tangentiale Abbildungsfehler. Diese führen dazu, dass die erfassten Pixelpositionen auf dem Bild nicht exakt den realen Positionen im Weltkoordinatensystem entsprechen, sondern leicht verschoben sind.

Um diese Bildverzerrung zu korrigieren, kann die Funktion `cv2.undistort()` aus der OpenCV-Bibliothek verwendet werden. Diese Funktion benötigt als Eingabeparameter die *Kameramatrix*, die *Verzerrungskoeffizienten* sowie das zu entzerrende Bild. Das Ergebnis ist ein geometrisch korrigiertes Bild, das die Grundlage für eine präzise Bildverarbeitung bildet.

Die **extrinsischen Parameter** $\mathbf{R}_{\text{vec}}$ (Rotationsvektor) und $\mathbf{t}_{\text{vec}}$ (Translationsvektor) beschreiben die räumliche Beziehung zwischen dem *Weltkoordinatensystem* (z. B. durch ein Schachbrett Kalibriermuster definiert) und dem *Kamerakoordinatensystem*. Diese Transformation besteht aus einer Rotation und einer darauffolgenden Translation:

$$\mathbf{x}_K = \mathbf{R} \cdot \mathbf{x}_W + \mathbf{t} [23]$$

Um diese Transformation kompakt in homogenen Koordinaten darzustellen, wird sie in Matrixform erweitert:

$$\begin{pmatrix} x_k \\ y_k \\ z_k \\ 1 \end{pmatrix} = \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_1 \\ r_{21} & r_{22} & r_{23} & t_2 \\ r_{31} & r_{32} & r_{33} & t_3 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x_w \\ y_w \\ z_w \\ 1 \end{pmatrix} = \tilde{\mathbf{R}}\mathbf{T} \cdot \tilde{\mathbf{x}}_W [23]$$

Dabei bezeichnet $\tilde{\mathbf{R}}\mathbf{T}$ die kombinierte Rotations-Translations-Matrix in homogener Form, und $\tilde{\mathbf{x}}_W$ den entsprechenden Punkt im Weltkoordinatensystem. Diese Beziehung bildet die Grundlage für die korrekte Abbildung von 3D-Punkten in der 2D-Kameraansicht. Die Rotationsmatrix kann mithilfe der Funktion `Rodrigues()` aus der OpenCV-Bibliothek berechnet werden. Diese Funktion konvertiert den Rotationsvektor, der die Rotation in kompakter Form beschreibt, in eine äquivalente 3×3 -Rotationsmatrix. Als Eingabeparameter wird ausschließlich der Rotationsvektor benötigt.

Eine direkte Verwendung dieser Transformation für die Transformation zwischen Kamerakoordinatensystem und Weltkoordinatensystem (welches während der Kalibrierung über das Schachbrettmuster definiert wird) wäre grundsätzlich möglich, um auf das globale Koordinatensystem zu schließen. Allerdings ergeben sich dabei mehrere Nachteile:

Zum einen liegen die Koordinaten im Kamerakoordinatensystem in einer willkürlichen Einheit vor. Es besteht keine eindeutige Skalierung in physikalischen Größen wie Metern oder Millimetern. Dies erschwert eine präzise Übertragung der Positionsdaten in ein anwendungsfreundliches Maßsystem.

Zum anderen bezieht sich diese Transformation auf ein kalibriertes Weltkoordinatensystem, das fest mit dem Schachbrettmuster verknüpft ist. Dieses stellt jedoch nicht das tatsächlich verwendete globale Koordinatensystem des Versuchsaufbaus dar. Um die Koordinaten korrekt in das gewünschte globale System zu überführen, wäre eine zusätzliche Transformation vom Schachbrettkoordinatensystem in das globale Koordinatensystem notwendig. Dies würde den Gesamtprozess erheblich verkomplizieren.

3.2.4 Berechnung der Transformationsmatrix

Da die Ansteuerung des Roboterarms ausschließlich über das globale Koordinatensystem auf dem Tisch erfolgt, muss zunächst die entsprechende Transformation vom Bildkoordinatensystem in das globale Koordinatensystem bestimmt werden. Ziel dabei ist es, die Bildkoordinaten (u, v) mittels einer geeigneten Koordinatentransformation in die entsprechenden globalen Koordinaten (x_G, y_G) zu überführen.

Ausgehend von diesen globalen Koordinaten lassen sich anschließend die gewünschten Positionen in die jeweiligen Koordinatensysteme der einzelnen Roboterarme transformieren, sodass eine präzise Steuerung und Positionierung möglich ist.

Zur Umrechnung der Koordinatensysteme bedarf es einer vorherigen Klärung, wie das Kamerasystem in Relation zum globalen Koordinatensystem positioniert wurde:

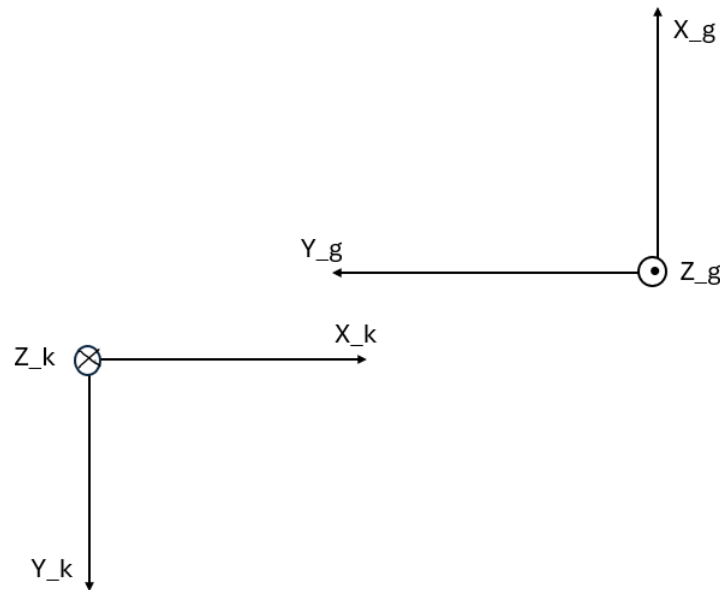


Abbildung 3.11: Position des globalen Koordinatensystems vs. des Kamerakoordinatensystems

Da das Koordinatensystem der Kamera parallel zum globalen Koordinatensystem ausgerichtet ist, können im Vorfeld bestimmte Annahmen getroffen werden, die die Transformation zwischen den beiden Koordinatensystemen vereinfachen.

- Das Zweidimensionale Koordinatensystem der Kamera (X_k, Y_k) liegt in der dreidimensionalen Ebene des globalen Koordinatensystems (X_g, Y_g, Z_g).
- Da die Kamera so ausgerichtet wurde, dass ihre Achsen parallel zum globalen Koordinatensystem verlaufen, kann die z -Koordinate des zweidimensionalen Kamerakoordinatensystems für die Transformation als konstant angenommen werden. Diese Koordinate hat einen festen Wert, der die Ausrichtung und Position der Kamera im Raum beschreibt.

Aus dieser Annahme kann abgeleitet werden, dass für alle dreidimensionalen Punkte $P_{3D}(x, y, z)$ die Höhe z im Operationsbereich konstant ist und somit vorab festgelegt werden kann. Damit lässt sich jeder zweidimensionale Bildpunkt $P_{2D}(x, y)$ aus dem Kamerakoordinatensystem als ein dreidimensionaler Punkt $P_{3D}(x', y', z)$ abgebildet werden kann, wobei z den festen vertikalen Abstand zwischen Kamera- und globalem Koordinatensystem beschreibt.

Es ist auch zu beachten, dass nicht das uEye Kamerasystem die Orientierung des Koordinatensystems vorzugeben hat, sondern die eingebundene Bibliothek OpenCV, bei welcher der Koordinatenursprung obligatorisch in der oberen linken Ecke des Bildes angesetzt ist.

Da die Transformationsmatrix aus einer Rotations- und einer Translationskomponente besteht, beschreibt der Translationsanteil die Verschiebung des Ursprungs des Kamerakoordinatensystems zum Ursprung des globalen Koordinatensystems. Daraus ergibt sich die folgende Transformationsgleichung:

$$T_{ges} = R_x(\theta) \cdot R_y(\alpha) \cdot R_z(\phi) \cdot T_{translation}$$

Der Koordinatenursprung des Kamerasystems wurde in der oberen linken Ecke des Bildes festgelegt. Die Translationswerte für die Achsen x und y werden durch einfache manuelle Messung

bestimmt. Die ermittelten Werte werden dokumentiert und in eine 4×4 Translationsmatrix übernommen:

$$\mathbf{T}_{\text{translation}} = \begin{pmatrix} 0 & 0 & 0 & t_x \\ 0 & 0 & 0 & t_y \\ 0 & 0 & 0 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Hinsichtlich der Rotationsmatrix wird zunächst die allgemeine Formel für eine Rotation um die möglichen Achsen aufgestellt:

$$\mathbf{R}_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(\theta) & -\sin(\theta) & 0 \\ 0 & \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}_y(\theta) = \begin{pmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$\mathbf{R}_z(\theta) = \begin{pmatrix} \cos(\theta) & -\sin(\theta) & 0 & 0 \\ \sin(\theta) & \cos(\theta) & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Die Abbildung 3.11 verdeutlicht, dass die betrachteten Koordinatensysteme unterschiedliche Achsenorientierungen aufweisen. Um diese auf eine gemeinsame Ausrichtung zu bringen, ist eine Rotation der z -Achse um 90° und eine Rotation der x -Achse um 180° , jeweils entgegen dem Uhrzeigersinn, erforderlich.

Daraus folgt, dass die Transformation zwischen den Koordinatensystemen lediglich diese beiden Rotationen sowie die anschließende Translation berücksichtigen muss. Die resultierende Transformationsgleichung lautet daher:

$$T_{\text{ges}} = R_x(180^\circ) \cdot R_z(90^\circ) \cdot T_{\text{translation}}$$

Im weiteren Verlauf der Transformation werden die Einheiten der beteiligten Koordinatensysteme vereinheitlicht. Dies erfolgte durch die Umrechnung der Bildkoordinaten von Pixel (px) in Zentimeter (cm), da das globale Koordinatensystem ausschließlich in Zentimetern definiert ist und die Steuerung des Roboters auf diesem Koordinatensystem basiert.

Dafür wurde ein Lineal als Referenz für die Distanz in der Bildebene von der Kamera koordinaten System auf den Höhenniveau des Operationstisches positioniert. Durch die Ausgabe der Koordinaten entlang eines Zentimeters konnte die Pixeldichte auf einer Geraden zur Umrechnung und Vereinheitlichung der Längeneinheiten in Gesamtsystem ermittelt werden. An dieser Stelle ist es wichtig darauf hinzuweisen, dass die Kamera in einer festen Höhe auf dem Operationsniveau positioniert sein muss. Wird die Höhe des Operationsbereichs verändert, so ändert sich auch das Verhältnis von Pixeln pro Zentimeter, was zu fehlerhaften Bestimmungen der globalen Koordinaten führt.

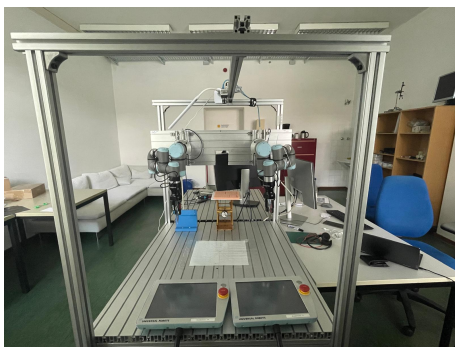
3.3 Beschreibung des experimentellen Aufbaus und der verwendeten Hardware

Dieser Abschnitt befasst sich mit der Integration und Installation der Hardware-Komponenten, die im Rahmen dieser Bachelorarbeit für experimentelle Versuche eingesetzt wurden. Zu den in das System integrierten Komponenten zählen:

- uEye CP Kamera
- Eine 50/60 Hz LED Lichtquelle
- Verwendes Lineal für handliche Messungen und Schachbrettmuster zur Kalibrierung der Kamera
- Beide Roboterarme in ihrem Arbeitsfeld

Der Grundaufbau des Versuchsfeldes umfasste die Installation zweier UR3e-Roboterarme, die senkrecht zum Operationsfeld an einer eigens konstruierten Aluminiumstruktur montiert werden. Diese Aluminiumkonstruktion umschließt den gesamten Arbeitsbereich der Roboterarme und ist so ausgelegt, dass sie an den Eckpunkten des zulässigen Arbeitsraums fixiert ist. Dadurch wird eine hohe strukturelle Steifigkeit und Robustheit gegenüber möglichen Vibrationen gewährleistet.

Die beiden Roboterarme werden in einem Abstand von etwa 55 cm zueinander installiert, wobei sich der Operationstisch exakt unterhalb der Roboteraufhängung befindet. Auf der Oberfläche des Operationstisches ist das globale Koordinatensystem markiert, das als Referenz für alle Positionierungen dient. Über dem Tisch befindet sich ein höhenverstellbares Hautimitat, dessen Position flexibel entlang aller Achsen angepasst werden kann, um unterschiedliche Szenarien zu simulieren.



(a)



(b)

Abbildung 3.12: Grundaufbau des Versuch: a) Roboteraufhängung des experimentellen Grundaufbaus mit verstellbarem Operationstisch im Vordergrund. b) uEye Kamera an oberer Querstrebe, aufgenommen im Labor für autonome Chirurgie an der Berliner Hochschule für Technik

Die Kamera wird oberhalb des Aktionsraums der beiden Roboterarme an einer stabilen Eisenstange befestigt, sodass sie eine vollständige Übersicht über die gesamte Hautattrappe hat. Dadurch kann sie die Erkennung der Nadel nach deren Austritt aus der Wunde ermöglichen.

Die Stromversorgung der Kamera erfolgt über USB, während die Kommunikation zwischen Kamera und Steuerrechner über ein Netzkabel (RJ45-Port) ermöglicht wird, wie bereits in Abschnitt 2.2 des zweiten Kapitels beschrieben.

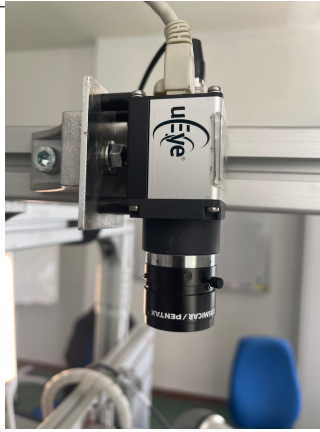


Abbildung 3.13: uEye CP Kamera

Die Implementierung einer geeigneten Beleuchtung stellte zu Beginn eine besondere Herausforderung dar. Ziel ist es, das Hautimitat optimal auszuleuchten, um die Erkennung der Nadel im Kamerabild zu erleichtern. Da die Nadel aus Aluminium besteht und somit stark reflektierend wirkt, kann durch gezielte Beleuchtung die Sichtbarkeit der Nadel zusätzlich verbessert werden: Reflexionen des Lichtstrahls an der Nadeloberfläche verstärken den Kontrast und erleichtern damit ihre Detektion.

Als Lichtquelle wurde eine LED mit 50/60 Hz Betriebsfrequenz gewählt, da diese konstant mit einer festen Frequenz betrieben wird und somit eine flimmerfreie Beleuchtung gewährleistet, was die Bildqualität und die Erkennungsgenauigkeit weiter verbessert.

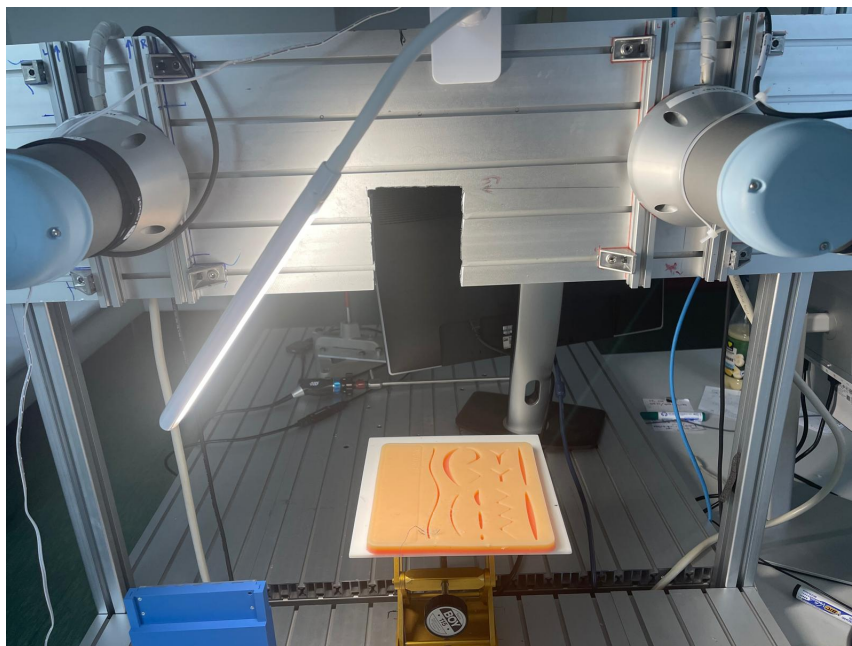


Abbildung 3.14: 50/60 Hz LED Lichtquelle

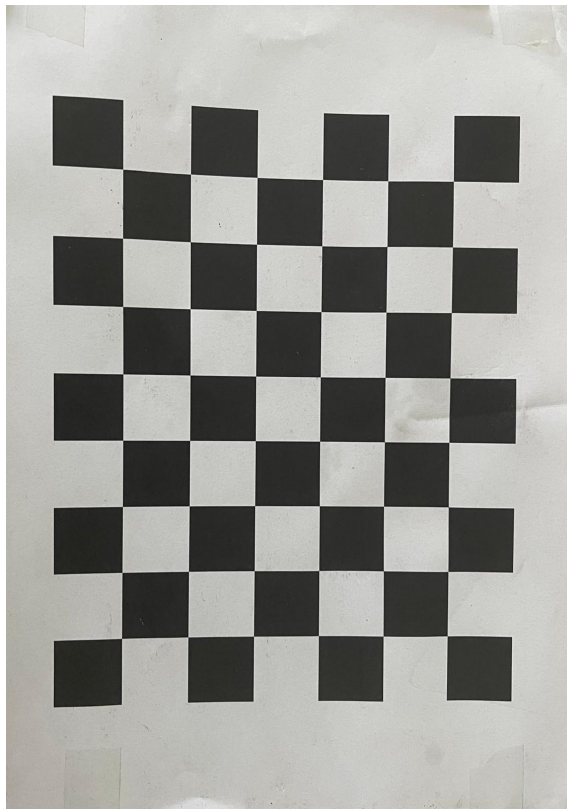
- **Schachbrettmuster für die Kamerakalibrierung**

Für die Kamerakalibrierung wird ein gedrucktes Schachbrettmuster eingesetzt, bestehend aus einem regelmäßigen Raster schwarzer und weißer Quadrate. Dieses Kalibriermuster ermöglicht die präzise Bestimmung der Kameraparameter durch die Detektion der Eck-

punkte der schwarzen Felder im Bild. Die Geometrie des Musters ist bekannt und bildet die Grundlage für die Ermittlung der intrinsischen Parameter, wie Brennweite, Bildmittelpunkt und Linsenverzerrung. Das hier verwendete Schachbrettmuster verfügt über eine Anordnung von 8 Ecken pro Reihe und 6 Ecken pro Spalte. Es wurde plan auf eine ebene Fläche aufgebracht, sodass eine eindeutige Zuordnung der Bildkoordinaten zu den bekannten Positionen im realen Raum erfolgen kann. Jedes Quadrat des Musters hat eine Kantenlänge von 30 mm.

- **Maßband zur Referenzmessung**

Für handliche Messungen und zur Bestimmung von Distanzen im Versuchsaufbau kam ein standardisiertes, metrisch kalibriertes Maßband (Gliedermaßstab) zum Einsatz. Das Maßband verfügt über eine gut ablesbare Skala in Zentimeter oder Millimeter-Schritten und wurde verwendet, um Entfernungen zwischen dem Ursprung des globalen Koordinatensystems und dem Bildkoordinatensystem zu ermitteln, sowie um die Menge der Pixel pro Zentimeter zu ermitteln. Die Messgenauigkeit des Maßbands liegt bei ± 1 mm und ist damit ausreichend für die vorliegende Anwendung in der experimentellen Kamerakalibrierung.



(a) Checkerboard-Muster für die Kalibrierung der Kamera



(b) Verwendete Lineal für die Messungen

Abbildung 3.15: Messmaterialien und Schachbrettmuster

3.4 Realisierung

In diesem Abschnitt wurde mit der Softwareentwicklung begonnen. Die hergeleiteten Formeln für die elliptische Bewegung, sowie der gesamte Prozess zur Erkennung der Nadel und des Fadens werden in der Programmiersprache Python implementiert.

3.4.1 Realisierung der elliptische Bewegung

Bei der Realisierung wird zunächst festgestellt wo die Nadel vor der Wunde einsteigt und wo die Nadel aus der Wunde herauskommt. Dies hängt von den Ellipsenparametern a und b ab.

```

15 def Elliptische_bewegung( robot_2, a, b, acc=0.02, vel=0.01):
16
17
18     # Beschleunigung und Geschwindigkeit des Roboters
19     acc = 0.2
20     vel = 0.2
21
22     # parameter der Ellipse für die Ermittlung der Radius
23     a = 0.0006
24     b = 0.001
25
26     # Radius der Nadel und HalteWinkel sowie die Versatz von Tcp zu Nadelspitze
27     rn = 0.002
28     theta = np.pi / 2
29     offset = np.array([0, -rn * np.sin(theta), rn * (np.cos(theta) - 1)]).reshape(3, 1)
30
31     # Berechnung alle punkte der Ellipse über variable phi winkel
32     for i in range(180, 270):
33         phi = i * np.pi / 180.0
34         r = (a * b) / np.sqrt((b ** 2 * np.cos(phi) ** 2) + (a ** 2 * np.sin(phi) ** 2))
35
36         # Ableitungen (Tangenten) : orientierung des TCP
37         dy_i1 = -r * np.sin(i * np.pi / 180.0) + np.gradient(r) * np.cos(i * np.pi / 180.0)
38         dz_i1 = r * np.cos(i * np.pi / 180.0) + np.gradient(r) * np.sin(i * np.pi / 180.0)
39
40         dy_i2 = -r * np.sin((i+1) * np.pi / 180.0) + np.gradient(r) * np.cos((i+1) * np.pi / 180.0)
41         dz_i2 = r * np.cos((i+1) * np.pi / 180.0) + np.gradient(r) * np.sin((i+1) * np.pi / 180.0)
42
43         deltax, deltay, deltaz = 0.0, r * (np.cos((i + 1) * np.pi / 180.0) - np.cos(i * np.pi / 180.0)), r * (
44             np.sin((i + 1) * np.pi / 180.0) - np.sin(i * np.pi / 180.0))
45         deltasalpha, deltabeta, deltagamma = 0.0, dy_i2-dy_i1, dz_i2-dz_i1
46
47         vf.change_robot_pose_delta(robot_2, delta_x=deltax, delta_y=deltay, delta_z=deltaz, delta_alpha=deltasalpha,
48             delta_beta=deltabeta, delta_gamma=deltagamma, acc_in=acc, vel_in=vel)

```

Abbildung 3.16: Implementierung der elliptischen Bewegung

Der in Abbildung 3.16 dargestellte Python-Code implementiert die Bewegung des Roboters entlang der elliptischen Bahn. Zu Beginn werden die Bewegungsparameter wie die Beschleunigung acc und die Geschwindigkeit vel definiert. Anschließend werden die Parameter der Ellipse festgelegt: die Halbachsen a und b , der Radius r_n sowie der Haltewinkel $\theta = \frac{\pi}{2}$.

Ein zentraler Bestandteil dieser Implementierung ist der Versatzvektor Δ_p , welcher die Abweichung zwischen dem Tool Center Point (TCP) des Roboters und einem Punkt auf der Ellipse beschreibt. Dieser Vektor wird einmalig zu Beginn aus dem Winkel θ berechnet und bleibt während der gesamten Bewegung konstant.

Die Berechnung der Punkte auf der Ellipse erfolgt innerhalb einer Schleife über 90 Schritte, wobei der Winkelbereich $\phi \in [180^\circ, 270^\circ]$ durchlaufen wird. Für jeden Schritt wird der Winkel

$$\phi = \frac{i \cdot \pi}{180}$$

berechnet und daraus der Radius r nach folgender Formel bestimmt:

$$r = \frac{a \cdot b}{\sqrt{b^2 \cdot \cos^2(\phi) + a^2 \cdot \sin^2(\phi)}}$$

Zur Berechnung der Bewegung des Roboters wird jeweils die Differenz zwischen zwei benachbarten Punkten auf der Ellipse verwendet. Die Differenzvektoren Δx , Δy und Δz ergeben sich dabei aus:

$$\begin{aligned} \Delta x &= r \cdot \left[\cos\left(\frac{(i+1) \cdot \pi}{180}\right) - \cos\left(\frac{i \cdot \pi}{180}\right) \right] \\ \Delta y &= r \cdot \left[\sin\left(\frac{(i+1) \cdot \pi}{180}\right) - \sin\left(\frac{i \cdot \pi}{180}\right) \right] \\ \Delta z &= 0 \end{aligned}$$

Da der Versatzvektor $\vec{\Delta}_p$ unabhängig vom Winkel ϕ ist, kürzt er sich bei der Berechnung der Punktdifferenz heraus. Die aktuelle TCP-Position des Roboters ergibt sich aus:

$$P_{tcp} = \vec{P}_t - \vec{\Delta}_p$$

Die Positionsänderungen, sowie eine Orientierung (bzw. Rotation um die y - und z -Achse um den Winkel $(\dot{r} \cdot \cos(\varphi) - r \cdot \sin(\varphi))$ in die y -Richtung und um den Winkel $(\dot{r} \cdot \sin(\varphi) + r \cdot \cos(\varphi))$ in die z -Richtung) werden über die Funktion `change_robot_pose_delta()` an den Roboter übergeben. Diese Funktion stammt aus der Vektorfunktions-Bibliothek und ermöglicht die schrittweise Steuerung des Roboters entlang der berechneten elliptischen Bahn.

3.4.2 Realisierung der Bildverarbeitung zur Erkennung von Nadel und Faden

Zur Realisierung des Bildverarbeitungsteils wurde zunächst die Kalibrierung der Kamera durchgeführt. Die Implementierung dieses Schritts in Python wird im Folgenden dargestellt:

```

5 # termination criteria
6 criteria = (cv.TERM_CRITERIA_EPS + cv.TERM_CRITERIA_MAX_ITER, 30, 0.001)
7
8 # prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....,(6,5,0)
9 objp = np.zeros((6*7,3), np.float32)
10 objp[:, :2] = np.mgrid[0:7,0:6].T.reshape(-1,2)
11
12 # Arrays to store object points and image points from all the images.
13 objpoints = [] # 3d point in real world space
14 imgpoints = [] # 2d points in image plane.
15
16 images = glob.glob('*.jpg')
17
18 for fname in images:
19     img = cv.imread(fname)
20     gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
21
22     # Find the chess board corners
23     ret, corners = cv.findChessboardCorners(gray, (7,6), None)
24
25     # If found, add object points, image points (after refining them)
26     if ret == True:
27         objpoints.append(objp)
28
29         corners2 = cv.cornerSubPix(gray, corners, (11,11), (-1,-1), criteria)
30         imgpoints.append(corners2)
31
32         # Draw and display the corners
33         cv.drawChessboardCorners(img, (7,6), corners2, ret)
34         cv.imshow('img', img)
35         cv.waitKey(500)
36
37 cv.destroyAllWindows()
38 ret, mtx, dist, rvecs, tvecs = cv.calibrateCamera(objpoints, imgpoints, gray.shape[::-1], None, None)

```

Abbildung 3.17: Kalibrierungsprogramm

Im vorangestellten Quellcode wird die Vorgehensweise zur Durchführung der Kamerakalibrierung beschrieben. Zunächst werden die Abbruchkriterien für die Optimierung auf Subpixel-Genauigkeit definiert, um die Genauigkeit der Eckendetektion zu erhöhen.

Anschließend werden zwei Arrays vorbereitet, um die Kalibrierungspunkte zu speichern:

- `objpoints` enthält die 3D-Koordinaten der bekannten Punkte im realen Raum (z. B. auf dem Schachbrettmuster).
- `imgpoints` speichert die zugehörigen 2D-Koordinaten dieser Punkte im Bild.

Für die 3D-Referenzpunkte des Schachbretts wird ein Array erstellt und mit Nullen initialisiert. Die x - und y -Koordinaten werden anschließend mit einem Gitterwert entsprechend der Schachbrettgröße befüllt. Dieser Array stellt die bekannten Referenzpunkte für die Kalibrierung dar. Daraufhin werden alle Kalibrierbilder aus einem definierten Verzeichnis geladen. Diese Bilder dienen als Grundlage für den Kalibrierprozess.

Für jedes Bild wird im nächsten Schritt eine Schleife durchlaufen: Zunächst wird das Bild eingelesen und in ein Graustufenbild umgewandelt. Anschließend erfolgt die Suche nach den Schachbrettecken mit der Funktion `cv2.findChessboardCorners()`.

Wenn Ecken erfolgreich erkannt werden, werden die entsprechenden 3D-Punkte in das vorbereitete Array übernommen. Die Positionen der gefundenen Ecken im Bild werden mithilfe von `cv2.cornerSubPix()` auf Subpixel-Ebene verfeinert. Diese verfeinerten 2D-Bildkoordinaten werden schließlich in `imgpoints` gespeichert und für die spätere Kalibrierung verwendet.

Zur visuellen Kontrolle wird das Ergebnis der Eckenerkennung am Bildschirm angezeigt. Abschließend erfolgt die eigentliche Kalibrierung: Mit den zuvor gesammelten 3D-Referenzpunkten und 2D-Bildpunkten wird `cv2.calibrateCamera()` aufgerufen. Diese Funktion berechnet automatisch die Kameramatrix, die Verzerrungskoeffizienten, den Rotationsvektor und den Translationsvektor.

Danach wird ein Skript entwickelt, das automatisiert ein Bild aufnimmt und dieses unter einem frei wählbaren Namen abspeichert. Die Auflösung der aufgenommenen Bilder wird dabei auf 1280 x 960 Pixel festgelegt.

```

1 def capture_image():
2     hcam = ueye.HIDS(0)
3     ret = ueye.is_InitCamera(hcam, None)
4     if ret != ueye.IS_SUCCESS:
5         print("ERROR: is_InitCamera failed")
6         return None
7
8     # set color mode
9     ret = ueye.is_SetColorMode(hcam, ueye.IS_CM_BGR8_PACKED)
10    if ret != ueye.IS_SUCCESS:
11        print("ERROR: is_SetColorMode failed")
12        return None
13
14    # set region of interest
15    width = 1280
16    height = 960
17    rect_aoi = ueye.IS_RECT()
18    rect_aoi.s32X = ueye.int(0)
19    rect_aoi.s32Y = ueye.int(0)
20    rect_aoi.s32Width = ueye.int(width)
21    rect_aoi.s32Height = ueye.int(height)
22    ret = ueye.is_AOI(hcam, ueye.IS_AOI_IMAGE_SET_AOI, rect_aoi, ueye.sizeof(rect_aoi))
23    if ret != ueye.IS_SUCCESS:
24        print("ERROR: is_AOI failed")
25        return None
26
27    # allocate memory
28    mem_ptr = ueye.c_mem_p()
29    mem_id = ueye.int()
30    bitspixel = 24 # for BGR8
31    ret = ueye.is_AllocImageMem(hcam, width, height, bitspixel, mem_ptr, mem_id)
32    if ret != ueye.IS_SUCCESS:
33        print("ERROR: is_AllocImageMem failed")
34        return None
35
36    # set memory
37    ret = ueye.is_SetImageMem(hcam, mem_ptr, mem_id)
38    if ret != ueye.IS_SUCCESS:
39        print("ERROR: is_SetImageMem failed")
40        return None
41
42    # capture single frame
43    ret = ueye.is_FreezeVideo(hcam, ueye.IS_WAIT)
44    if ret != ueye.IS_SUCCESS:
45        print("ERROR: is_FreezeVideo failed")
46        return None
47
48    # convert image memory to numpy array
49    lineinc = width * int((bitspixel + 7) / 8)
50    try:
51        img = ueye.get_data(mem_ptr, width, height, bitspixel, lineinc, copy=True)
52        img = np.reshape(img, (height, width, 3))
53    except Exception as e:
54        print(f"ERROR while getting image data: {e}")
55        return None
56
57    # cleanup
58    ueye.is_FreeImageMem(hcam, mem_ptr, mem_id)
59    ueye.is_ExitCamera(hcam)
60
61    return img

```

Abbildung 3.18: Code für automatische Aufnahme eines Bildes

Nachdem die Nadel und der Bildfehler erkannt wurde, erfolgt anschließend die Implementierung in die Transformationsmatrix.


```

18 def Transform_from_kamera_to_global(Pixel_corr, rot_x_angle = 180, rot_y_angle = 0, rot_z_angle=90):
19
20     pixel_per_cm = 50
21     acc = 0.2
22     vel = 0.2
23
24     rot_x_angle = np.radians(rot_x_angle)
25     rot_y_angle = np.radians(rot_y_angle)
26     rot_z_angle = np.radians(rot_z_angle)
27
28
29     x_translation = -20.7 |
30     y_translation = 16.3
31     z_translation = 25
32
33
34
35     R_x = np.array([[1,0,0,0],
36                    [0, np.cos(rot_x_angle), -np.sin(rot_x_angle), 0],
37                    [0, np.sin(rot_x_angle), np.cos(rot_x_angle), 0],
38                    [0, 0, 0, 1]])
39
40     R_y = np.array([[np.cos(rot_y_angle), 0, np.sin(rot_y_angle), 0],
41                    [0, 1, 0, 0],
42                    [-np.sin(rot_y_angle), 0, np.cos(rot_y_angle), 0],
43                    [0, 0, 0, 1]])
44
45     R_z = np.array([[np.cos(rot_z_angle), -np.sin(rot_z_angle), 0, 0],
46                    [np.sin(rot_z_angle), np.cos(rot_z_angle), 0, 0],
47                    [0, 0, 1, 0],
48                    [0, 0, 0, 1]])
49
50     T = np.array([
51         [1, 0, 0, x_translation],
52         [0, 1, 0, y_translation],
53         [0, 0, 1, z_translation],
54         [0, 0, 0, 1]
55     ])
56
57     T_ges = R_x @ R_z @ T
58     coordinates_cm = np.array([
59         Pixel_corr[0] / pixel_per_cm,
60         Pixel_corr[1] / pixel_per_cm, # 51
61         0,
62         1
63     ])
64
65     #transformed_coords = R @ coordinates_cm + t
66     transformed_coords = (T_ges @ coordinates_cm)
67     transformed_coords = transformed_coords[:3]
68     print(T_ges)
69     print(transformed_coords)
70     return transformed_coords

```

Abbildung 3.19: Transformationsmatrix

Diese Transformationsmatrix ist notwendig, um das Bildkoordinatensystem an das globale Koordinatensystem anzupassen. Wie in Abschnitt 3.2.4 bereits erläutert, wird eine Drehung von 90° um die z -Achse und Anschließend eine Drehung von 180° um die x -Achse durchgeführt, damit die beide Koordinatensysteme übereinstimmen.

Zusätzlich wird die Translationsmatrix durch manuelle Messung des Verschiebungsvektors bestimmt, welcher den Abstand zwischen den Ursprüngen des Kamera- und des globalen Koordinatensystems entlang der x -, y - und z -Achse beschreibt. Die gemessenen Abstände betragen dabei etwa $-20,7$ cm in x -Richtung, $16,3$ cm in y -Richtung und 25 cm in z -Richtung.

Auch wenn diese Werte nicht absolut exakt sind, wurde bei der Messung eine hohe Genauigkeit angestrebt, um sicherzustellen, dass der Roboterarm die Nadel mit möglichst geringer Abweichung greifen kann.

Der nächste Schritt besteht darin, die Bilddifferenzen zu erzeugen. Hierfür wird eine Funktion implementiert, die als Eingabeparameter zwei Bilder benötigt: eines ohne Nadel und eines mit Nadel. Zusätzlich wird der Einstichpunkt der Nadel übergeben, welcher als wesentlicher Referenzpunkt dient, um den Suchbereich für die Nadelerkennung einzugrenzen.

```

def pixel_coordination(capture_1: np.array, capture_2: np.array, wound_entry_pos):
    # Camera Matrix aus der Kalibrierung
    mtx = np.array([[3.28187994e+03, 0.00000000e+00, 6.72438656e+02],
                    [0.00000000e+00, 3.28204211e+03, 4.62155433e+02],
                    [0.00000000e+00, 0.00000000e+00, 1.00000000e+00]])

    # Verzerrungskoeffizient aus der Kalibrierung
    dist = np.array([[-1.72721330e-03, 6.34657974e+00, 9.58632285e-03, 5.82966940e-03, -7.90004110e+01]])

    uint_img1 = np.array(capture_1).astype('uint8')
    img1 = cv2.cvtColor(uint_img1, cv2.COLOR_BGR2GRAY)
    uint_img2 = np.array(capture_2).astype('uint8')
    img2 = cv2.cvtColor(uint_img2, cv2.COLOR_BGR2GRAY)
    cv2.imshow('img', img2)
    cv2.waitKey(100)

    if img2 is None:
        print("Bild konnte nicht geladen werden!")

    # Bilder entzerren
    img1 = cv2.undistort(img1, mtx, dist)
    img2 = cv2.undistort(img2, mtx, dist)

    img1_resized = cv2.resize(img2, (img1.shape[1], img1.shape[0]))

    # Einstichpunkt des Nadels
    point_1 = np.array([-25, 0, 0])

    # Inverse Transformation globaler Koordinaten zum Bildkoordinaten
    inverse_transf = np.array([[0, -50, 215, 0],
                               [-50, 0, -1805, 0],
                               [0, 0, 25, 0],
                               [0, 0, 0, 1]])

    pixel_bereich = inverse_transf @ point_1
    pixel_bereich[0] - 20
    abs_diff = cv2.absdiff(img1, img2_resized)

    # Suche Bereich begrenzen
    abs_diff[(int(pixel_bereich[1]))-4:(int(pixel_bereich[1]))+100] - 0 # oben link
    abs_diff[(int(pixel_bereich[1]))+900:(int(pixel_bereich[1]))+100] - 0 # unten link
    abs_diff[(int(pixel_bereich[1]))+4:(int(pixel_bereich[1]))+100] - 0 # unten rechts
    abs_diff[(int(pixel_bereich[1]))+5:(int(pixel_bereich[1]))+100] - 0 # oben rechts
    abs_diff[(int(pixel_bereich[1]))+5:(int(pixel_bereich[1]))+5, int(pixel_bereich[1]))+1200] - 0 # klein stück recht
    abs_diff[(int(pixel_bereich[1]))+5:(int(pixel_bereich[1]))+5, int(pixel_bereich[1]))+1200] - 0 # klein stück link
    abs_diff[(int(pixel_bereich[1]))+5, int(pixel_bereich[1]))+100:(int(pixel_bereich[1]))+100] - 0

    _, mask = cv2.threshold(abs_diff, 10, 255, cv2.THRESH_BINARY)
    # Koordinaten in pixel extrahieren
    y_coords, x_coords = np.where(mask == 255)

    # Koordinaten zu (x,y)
    coordinates = list(zip(x_coords, y_coords))

    print('Anzahl unterschiedlicher Pixel: ', len(coordinates))

    print(coordinates[0])
    print(coordinates[-1])
    Mittel = (coordinates[0][0]+coordinates[-1][0])/2
    print(Mittel)
    Mittel_punkt = np.array([Mittel, coordinates[-1][1]])
    #print(Mittel_punkt)
    plt.figure(figsize=(12,6))

    plt.subplot(1,2,1)
    plt.title('Absolute Differenz')
    plt.imshow(abs_diff, cmap= 'gray')
    plt.axis('off')
    plt.show()

    return Mittel_punkt

```

Abbildung 3.20: Funktion zur Nadeldetektion

Die entwickelte Funktion nutzt die inverse Transformation der zuvor bestimmten Transformationsmatrix, um die Pixelkoordinaten des Einstichpunkts der Nadel im Bild zu berechnen. Basierend auf dieser Information wird die ROI festgelegt, in der die Nadel gezielt gesucht werden soll.

Zur Filterung der Bilddaten kommt die Funktion `cv2.threshold()` zum Einsatz. Dabei werden alle Pixel mit einem Wert kleiner als der gewählte Schwellwert (hier 10) verworfen, während Pixel mit höheren Werten auf 255 gesetzt werden. Durch den Einsatz einer zusätzlichen Lichtquelle wird die Nadel besonders stark beleuchtet, sodass ihre Pixelwerte den Schwellwert überschreiten und somit zuverlässig erkannt werden können. Pixel des Hintergrunds hingegen liegen unterhalb des Schwellwerts und werden ausgeblendet.

Nach der Segmentierung werden die erkannten Pixelkoordinaten extrahiert und als (x, y) -Koordinaten gespeichert. Anschließend wird der geometrische Mittelpunkt der erkannten Nadel berechnet. Die Bestimmung dieses Mittelpunkts ist entscheidend, da der Roboterarm so in die Lage versetzt wird, präzise im Zentrum des erkannten Nadelsegments zuzugreifen.

Der gesamte Prozess wird nun in einer Funktion gebündelt, um die Nadelerkennung und die darauf folgende Steuerung des Roboters vollständig zu automatisieren.

```

100 def nadel_detection(robot_1, capture_1, capture_2, wound_entry_pos):
101     Mittel_punkt = pixel_coordinates(capture_1, capture_2, wound_entry_pos)
102     print(Mittel_punkt)
103     transformed_coords = Transform_from_kamera_to_Global(pixel_coords[Mittel_punkt[0]], Mittel_punkt[1])
104     # ret_x_angle=180, ret_y_angle=0, ret_z_angle=90
105
106     print(transformed_coords)
107     position_1 = [transformed_coords[0] + 0.007, transformed_coords[1], 0.2462] #
108
109     # der Roboter fährt zur Nadel Mittel mit einem Abstand von 0.7cm in x- Richtung
110     vf.change_robot_position_abs(robot=robot_1,
111                                robot_mv=1,
112                                x=position_1[0],
113                                y=position_1[1],
114                                z=position_1[2],
115                                acc_in=0.01,
116                                vel_in=0.05,
117                                move_type="move1")
118
119     #justieren in y- Richtung
120     vf.change_robot_position_delta(robot_1, delta_y=-0.0008)
121
122     # der Roboter kommt zu einer optimal Greifposition
123     vf.change_robot_pose_delta(robot_1, delta_gamma=-1.57, acc_in=0.01, vel_in=2)
124
125     # der Roboter fährt zur Nadel
126     vf.change_robot_position_delta(robot_1, delta_x=-0.008)
127

```

Abbildung 3.21: Funktion zur Automatisierung des Prozesses und Steuerung des Roboters

Die ermittelten Pixelkoordinaten des Mittelpunkts der Nadel werden zunächst an die Funktion `transformation_from_kamera_to_Global` übergeben. Diese berechnet daraus die Position des erkannten Nadelmittlepunkts im globalen Koordinatensystem in Zentimetern.

Anschließend wird der Roboter mit Hilfe der Funktion `change_robot_abs()` zu diesem Punkt bewegt, wobei zusätzlich eine Offset-Korrektur von 0,7 cm in x -Richtung berücksichtigt wird. Danach erfolgt eine Feinjustierung um $-0,8$ mm in y -Richtung mittels der Funktion `change_robot_position_delta()`.

Schließlich wird die Orientierung des Roboterarms angepasst, um eine optimale Greifposition zu gewährleisten. Sobald diese Ausrichtung erreicht ist, fährt der Roboterarm kontrolliert zur erkannten Nadel und führt den Greifvorgang durch.

```

25 def pixelkoordinaten(capture_1: np.array, capture_2: np.array):
26
27     mtx = np.array([[3.283794e+03, 0.000000e+00, 6.274185e+04],
28                   [0.000000e+00, 3.3220411e+03, 6.2215451e+04],
29                   [0.000000e+00, 0.000000e+00, 1.000000e+00]])
30
31     dist = np.array([-1.7272133e-01, 6.3465707e+00, 9.5853225e-01, 5.8306504e-03, -7.5004113e+01])
32
33     img1 = cv2.imread(path1, cv2.IMREAD_GRAYSCALE).copy()
34     uint_img1 = np.array(capture_1).astype('uint8') # kein "255" eig, kommt schon so vom ukye
35     img1 = cv2.cvtColor(uint_img1, cv2.COLOR_BGR2GRAY)
36     cv2.imshow('img1', img1)
37     cv2.waitKey(100)
38
39     uint_img = np.array(capture_2).astype('uint8') # kein "255" eig, kommt schon so vom ukye
40     img2 = cv2.cvtColor(uint_img, cv2.COLOR_BGR2GRAY)
41     cv2.imshow('img2', img2)
42     cv2.waitKey(100)
43
44     if img2 is None:
45         print("Bild konnte nicht geladen werden!")
46
47     # Bild1: monoscan
48     img3 = cv2.undistort(img1, mtx, dist)
49     img2 = cv2.undistort(img2, mtx, dist)
50
51     cv2.imshow('img', img3)
52     cv2.waitKey(100)
53     img2_resized = cv2.resize(img2, (img1.shape[1], img1.shape[0]))
54
55     # einsteig Punkt
56     point_1 = np.array([-29, -0.8, 1, 0])
57
58     inverse_transf = np.array([[0, -50, 815, 0],
59                               [-50, 0, -1070, 0],
60                               [0, 0, 25, 0],
61                               [0, 0, 0, 1]])
62
63     pixel_bereich = inverse_transf@point_1
64
65     print(pixel_bereich)
66     abs_diff = cv2.absdiff(img1, img2_resized)
67     abs_diff[0:960, 0: int(pixel_bereich[0]-2)] = 0 # oben links
68     abs_diff[0: int(pixel_bereich[1]), 5: int(pixel_bereich[0]-2):1280] = 0 # unten links
69     abs_diff[0: int(pixel_bereich[2]):960, int(pixel_bereich[0]-2):1280] = 0 # unten rechts
70     abs_diff[0: int(pixel_bereich[0])+2:1280] = 0 # oben rechts
71     abs_diff[0:960, int(pixel_bereich[0])+3:1280] = 0 # klein stück rechts
72
73     hist1 = cv2.calcHist([img1], [0], None, [256], [0, 256])
74     hist2 = cv2.calcHist([img2_resized], [0], None, [256], [0, 256])
75     hist_diff = cv2.compareHist(hist1, hist2, cv2.HISTCMP_BHATTACHARYYA)
76
77     _mask = cv2.threshold(abs_diff, 10, 255, cv2.THRESH_BINARY)
78     #koordinaten in pixel extrahieren
79
80     y_coords, x_coords = np.where(_mask == 255)
81
82     # kombinieren zu (x,y)
83     coordinates = list(zip(x_coords, y_coords))
84
85     print('Anzahl unterschiedlicher Pixel:', len(coordinates))
86     print(coordinates[0])
87     print(coordinates[-1])
88     Mittel = (coordinates[0][0]+coordinates[-1][0])/2
89     print(Mittel)
90     Mittel_punkt = np.array([Mittel, coordinates[-1][1]])
91
92     plt.figure(figsize=(12,6))
93
94     plt.subplot(1,2,1)
95     plt.title('Absolute Differenz')
96     plt.imshow(abs_diff, cmap='gray')
97     plt.axis('off')
98     plt.show()
99
100     return Mittel_punkt

```

Abbildung 3.22: Funktion zur Fadendetektion

Das Prinzip der Fadendetektion basiert auf demselben Verfahren wie die Nadeldetektion. Der wesentliche Unterschied besteht jedoch darin, dass die Position des Fadens bereits bekannt ist: Er befindet sich exakt am Eintrittspunkt der Nadel. An dieser Stelle soll der Roboter gezielt den Faden greifen. Daher wird das Suchgebiet (ROI) gezielt auf den Eintrittsbereich der Nadel sowie auf einen kleinen Bereich von wenigen Millimetern links und rechts davon begrenzt. Die verwendete Funktion `transformation_from_kamera_to_Global` bleibt dabei unverändert.

```

114 def pfaden(robot_2, capture_1, capture_3, wound_entry_pos):
115     acc = 0.2
116     vel = 0.2
117     Mittel_punkt = pixelpfaden koordinaten(capture_1, capture_3) #'/home/frankheubel/Bilder/capture0.jpg'
118     print(Mittel_punkt)
119     transformed_coords = Transform_from_kamera_to_global(Pixel_coord=[Mittel_punkt[0], Mittel_punkt[1]],
120                                                         rot_x_angle=180, rot_y_angle=0, rot_z_angle=90)
121     print(transformed_coords)
122     position_1 = (transformed_coords[0], transformed_coords[1] - 0.0, 0.250) # 0.2476, #0.2469 #0.003
123     print(position_1)
124
125     robot_2 = initialize_robots(rob1=False, rob2=True, acc=acc, vel=vel, init_home_pos=False)
126
127     vf.change_robot_position_abs(robot=robot_2,
128                                robot_nr=2,
129                                x=position_1[0],
130                                y=position_1[1],
131                                z=position_1[2],
132                                acc_in=0.01,
133                                vel_in=0.05,
134                                move_type="move1")
135     vf.change_robot_position_delta(robot_2, delta_x=-0.0030)

```

Abbildung 3.23: Funktion zur Automatisierung des Prozesses und Steuerung des Roboters

Kapitel 4

Testmethoden und Ergebnisse

In diesem Kapitel werden die durchgeführten Testmethoden zur Evaluierung der Genauigkeit und Funktionalität des Systems vorgestellt. Zunächst wird die Testmethode und die zugehörigen Ergebnisse der kontinuierlichen elliptischen Bewegung beschrieben. Anschließend erfolgt die Darstellung der Testmethodik und deren Ergebnisse zur Evaluierung der IBVS-basierten Nadelerkennung.

4.1 Kontinuierliche elliptische Bewegung: Testmethodik und Ergebnisse

Wie bereits im Kapitel zur Realisierung beschrieben, soll die Nadel so geführt werden, dass sie etwa 1 mm vor der Wunde in das Gewebe eindringt und ca. 1 mm hinter der Wunde wieder austritt. Dieses Verhalten hängt maßgeblich von den Ellipsenparametern a und b , sowie von der Startorientierung des Roboters ab, d. h. von der Orientierung, die der Roboter zu Beginn der Bewegung einnimmt.

Zur Evaluation werden verschiedene Werte für die Parameter a und b und die Startorientierung getestet. Für jede Parameterkombination wird der resultierende Fehler im Hinblick auf die gestellte Anforderung (exakte Positionierung der Einstich- und Austrittspunkte) bestimmt und analysiert.

a [mm]	b [mm]	Startorientierung des TCP	Abweichung Austritt [mm]	Bewertung
5	3	90° Zur Wunde	10	zu groß
5	3	80° Zur Wunde	8	zu groß
5	3	70° Zur Wunde	5	groß
5	3	60° Zur Wunde	1	akzeptabel
5	3	50° Zur Wunde	0.8	gut
6	2	90° Zur Wunde	8	zu groß
6	2	80° Zur Wunde	8	zu groß
6	2	60° Zur Wunde	1	akzeptabel
6	2	50° Zur Wunde	0	gut
4	3	70° Zur Wunde	5	groß
4	3	90° Zur Wunde	9	zu groß
4	3	80° Zur Wunde	8	zu groß

Tabelle 4.1: Testtabelle der elliptischen Bewegung und Ergebnisse

Die Ergebnisse in der Abbildung 4.1 zeigen, dass große Abweichungen ,vor allem bei Startwinkeln von 80° und 90° auftreten, unabhängig von den verwendeten Ellipsenparametern. Besonders

geringe Abweichungen (unter 1 mm) konnten bei Startwinkeln von 50° und 60° erzielt werden. Diese Werte werden in der Bewertung als *gut* bzw. *akzeptabel* eingestuft. Deutlich höhere Abweichungen über 5 mm gelten als *groß* bzw. *zu groß* und würden in der Praxis zu einer ungenauen Positionierung führen.

Insgesamt lässt sich feststellen, dass die Wahl der Startorientierung einen größeren Einfluss auf die Genauigkeit hat als die Variation der Ellipsenparameter a und b .

Die Startorientierung des TCP, sowie die Abweichung am Austrittspunkt stellen keine exakten Messwerte dar. Da eine präzise Messung in diesem Fall nur schwer umsetzbar war, werden stattdessen Schätzwerte verwendet, die auf visueller Beobachtung basieren.

4.2 IBVS-Nadelerkennung: Testmethodik und Ergebnisse

Zur Evaluierung der Genauigkeit des Bildverarbeitungsmoduls wurde zunächst die Transformationsgenauigkeit vom Bildkoordinatensystem in das globale Koordinatensystem überprüft. Die angewandte Methode bestand darin, einen definierten Punkt (Pixelkoordinate) im aufgenommenen Bild auszuwählen und diesen mittels der ermittelten Transformationsmatrix in das globale Koordinatensystem zu überführen.

Anschließend wird der TCP des Roboters gezielt zu der entsprechenden Position im globalen Koordinatensystem versetzt. Dadurch kann überprüft werden, ob der Roboter den Zielpunkt exakt ansteuert oder ob es seitliche Abweichungen (z. B. nach links oder rechts) gibt. Auf diese Weise lässt sich die Genauigkeit und Zuverlässigkeit der Transformationsmatrix quantifizieren.

Um die Aussagekraft der Messung zu erhöhen und zufällige Fehler auszuschließen, wurde der Vorgang insgesamt 100-mal mit beiden Roboterarmen wiederholt. Die resultierenden Abweichungen dienen als Grundlage zur Bewertung der Präzision der Bild-zu-Welt-Koordinatentransformation. Die Ergebnisse dieser Testreihe werden im Folgenden dargestellt:

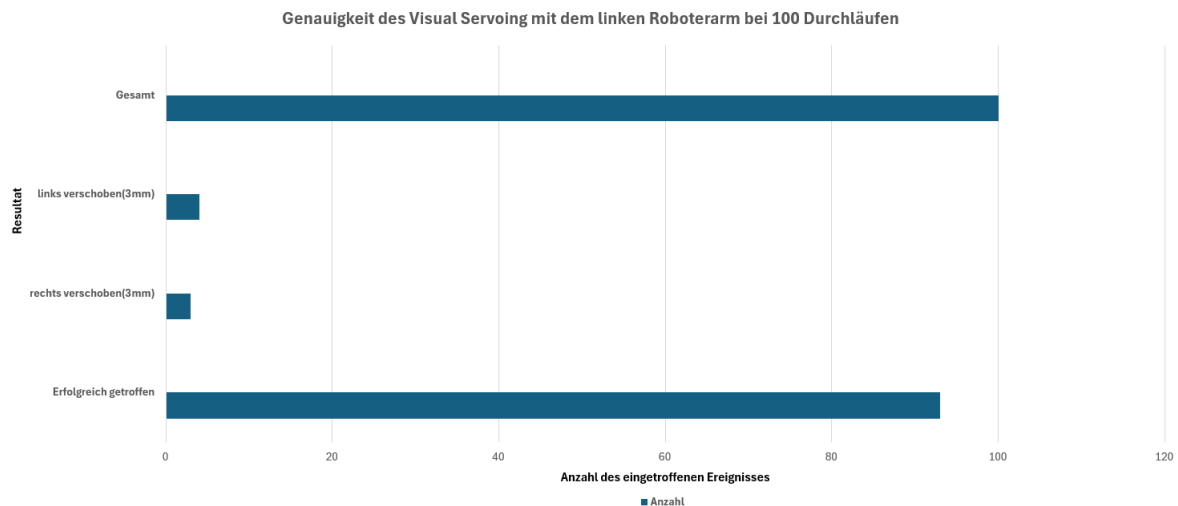


Abbildung 4.1: Ergebnisse der Visual-Servoing-Prüfung des linken Arms in 100 Testläufen

Die Grafik zeigt die Genauigkeit des linken Roboterarms beim Visual Servoing über 100 Testläufe hinweg. Dabei erreichte der Roboter in 93 % der Fälle exakt den Zielpunkt. In 4 % der Versuche wich der TCP um 3 mm nach rechts ab, in weiteren 3 % nach links. Diese Abweichungen werden

als geringfügig bewertet und sind in der Gesamtanalyse berücksichtigt.

Dasselbe Verfahren wurde auch auf den rechten Roboterarm angewendet. Die Ergebnisse werden nachfolgend dargestellt.

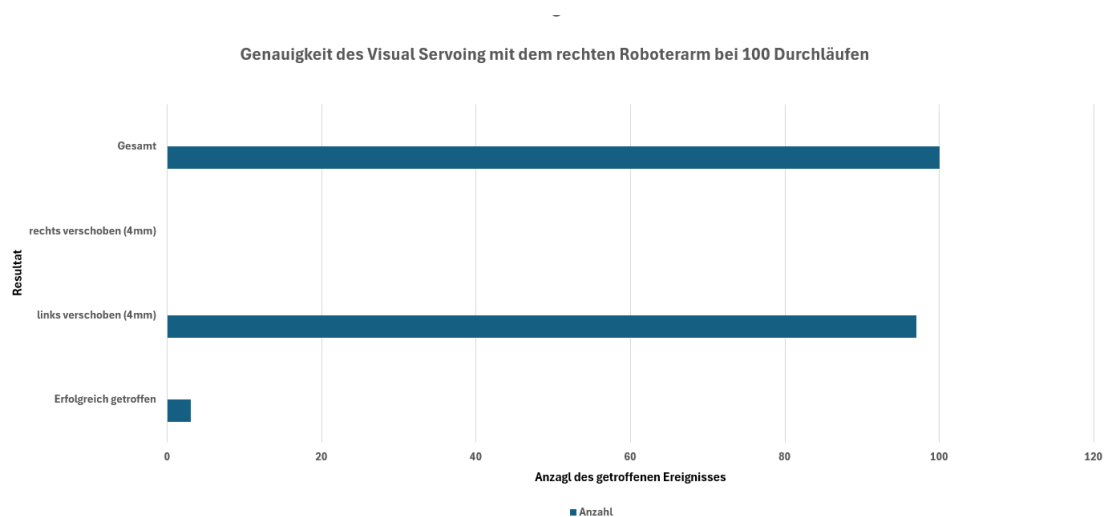


Abbildung 4.2: Ergebnisse der Visual-Servoing-Prüfung des linken Arms in 100 Testläufen

Die Grafik zeigt die Genauigkeit des Rechten Roboterarms beim Visual Servoing über 100 Testläufe hinweg. In 4 % der Fälle konnte der Roboter den Zielpunkt exakt treffen. In 96 % der Durchläufe wich der Tool Center Point (TCP) um 4 mm nach links ab.

Die beobachtete Linksabweichung lässt sich auf eine unzureichende Kalibrierung des Roboterarms 1 zurückführen. Beim Anfahren der Ausgangsposition im globalen Koordinatensystem wurde eine kleine systematische Verschiebung in y -Richtung festgestellt. Diese Abweichung wirkt sich auf die Genauigkeit der Ansteuerung aus und sollte in zukünftigen Kalibrierprozessen berücksichtigt werden. Insgesamt werden die Abweichungen als gering bewertet und in die Ergebnisanalyse einbezogen.

Nachdem die Genauigkeit beider Roboterarme einzeln überprüft worden ist, wird im nächsten Schritt untersucht, inwieweit der Roboterarm in der Lage ist, die Nadel sowie den Nahtpfaden präzise anzusteuern. Theoretisch lässt sich ableiten, dass wenn der Roboter definierte Zielpunkte zuverlässig erreichen kann, er ebenso in der Lage sein, die erkannte Position der Nadelspitze gezielt anzufahren. Im Folgenden wird daher zunächst analysiert, wie sich die Nadel und der Nahtpfaden im Differenzbild darstellen.

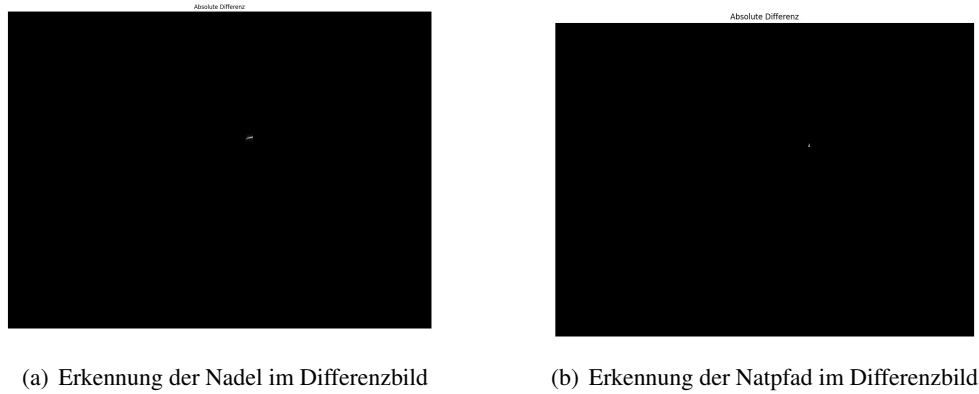


Abbildung 4.3: Erkennung der Nadel und des Fadens

Im linken Bild (a) ist die Nadel als heller Punkt in einem ansonsten schwarzen Bildbereich sichtbar. Das Differenzbild zeigt ausschließlich Bildänderungen, die zwischen zwei aufeinanderfolgenden Aufnahmen auftreten. Da die Nadel das einzige sich unterscheidende Objekt darstellt, werden unveränderte Bildanteile, etwa der Hintergrund ausgeblendet und erscheinen vollständig schwarz.

Im rechten Bild (b) ist der Faden im Bereich des Einstichpunkts der Nadel zu erkennen. Die Detektion erfolgt durch die kontinuierliche Differenzbildung zwischen zwei Einzelbildern, einem Referenzbild ohne Faden und einem Bild, in dem der Faden bereits vorhanden ist. Dadurch hebt sich der Faden als heller Streifen vom Hintergrund ab. Der übrige Bildbereich bleibt dunkel, was störende Einflüsse reduziert und eine gezielte Erkennung des Fadens ermöglicht. .

Zum Zweck des Testens wurde der Roboterarm insgesamt 50-mal gezielt zur Nadelspitze bewegt. Da für diese Aufgabe der linke Roboterarm (Roboterarm 1) verwendet wurde und bei vorherigen Tests eine systematische Abweichung von 4 mm nach links festgestellt wurde, wurde diese Abweichung durch eine manuelle Korrektur in y -Richtung (+4 mm) ausgeglichen.

Die Ergebnisse dieser 50 Testdurchläufe werden in der folgenden Grafik dargestellt:

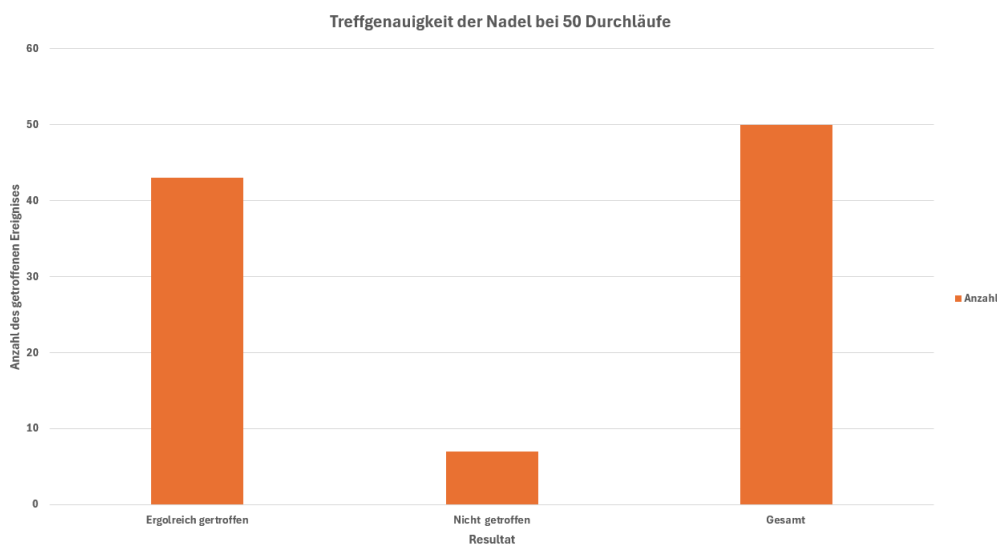


Abbildung 4.4: Visual-Servoing: Ergebnisse beim Nadelzugriff in 50 Testläufen

Die in Abbildung 4.3 dargestellte Grafik zeigt die Ergebnisse von 50 automatisierten Versuchen, bei denen der Roboterarm die aus der Wunde herausragende Nadelspitze gezielt greifen sollte. Ziel dieses Tests ist es, die Genauigkeit des Systems nach erfolgter Kalibrierung und Justierung des linken Roboterarms zu bewerten.

In 86 % der Fälle (43 von 50 Versuchen) konnte die Nadel erfolgreich gegriffen werden. In 14 % der Durchläufe (7 von 50 Versuchen) kam es zu einem Fehlschlag. Eine mögliche Ursache dafür liegt darin, dass bei wiederholten Tests an derselben Einstichstelle das Gewebe leicht beschädigt oder verformt wurde. Dies hat zu einer instabilen Lage der Nadel geführt, wodurch der Roboterarm die Greifposition nicht exakt anfahren kann. Zusätzlich erschwert ein leichtes Wackeln der Nadelspitze die erfolgreiche Erfassung.

Trotz dieser vereinzelt Abweichungen belegt die Grafik die hohe Präzision des Systems, sowie die Effektivität der vorgenommenen Justierung um 4 mm in y-Richtung, mit der die zuvor festgestellte systematische Abweichung des linken Roboterarms kompensiert wurde.

Das gleiche Verfahren wird angewendet, um die Genauigkeit bei der Anfahrt zum Faden zu evaluieren. Auch hierzu wird der Prozess insgesamt 50-mal wiederholt. Die anschließende Auswertung wird in der folgenden Abbildung graphisch dargestellt, womit sich die Präzision der Positionierung im Bezug auf den Nahtfaden anschaulich beurteilen lässt:

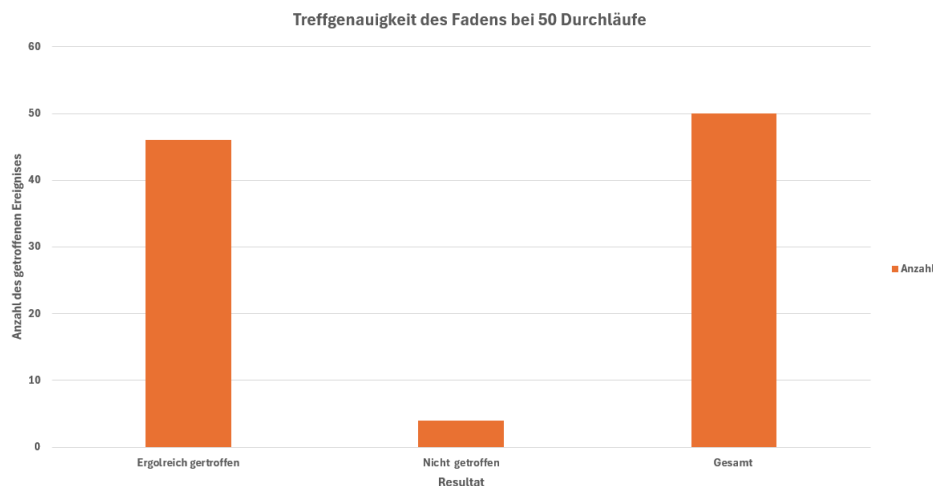


Abbildung 4.5: Visual-Servoing: Ergebnisse beim Fadenzugriff in 50 Testläufen

Die Auswertung der Grafik zeigt, dass in 94 % der Fälle (47 von 50 Versuchen) der Faden erfolgreich gegriffen werden konnte. In 6 % der Durchläufe (3 Versuche) kam es zu einem Fehlschlag. Dieses Ergebnis ist insgesamt besser als beim Nadelzugriff. Ein möglicher Grund hierfür ist, dass der Faden – im Gegensatz zur Nadel – auch nach mehrmaliger Nutzung desselben Einstichpunkts seine Lage beibehält und dadurch leichter zu greifen ist. Bei der Nadel führte die wiederholte Nutzung eines Einstichpunkts dagegen zu Gewebeschädigungen, die die Erkennung und das präzise Greifen erschweren. Hier lassen sich die Fehlschläge vermutlich darauf zurückführen, dass der Faden in diesen Fällen nicht in einer leicht greifbaren Position lag. Dafür spricht, dass der Greifer des Roboters den Faden nicht zuverlässig erfassen konnte, wenn dieser auf der Wunde lag und nicht nach oben hin ausgerichtet war.

Kapitel 5

Diskussion

5.1 Diskussion der Ergebnisse

Die Implementierung des Systems hat gezeigt, dass eine visuelle Steuerung des UR3e-Roboters und eine kontinuierliche Bewegung innerhalb der Wunde grundsätzlich realisierbar sind. Die Datenerfassung zur Nadelerkennung mit der uEye-CP-Kamera, sowie die Verarbeitung über die verschiedenen Koordinatensysteme lieferten zuverlässige Ergebnisse. Dennoch bestehen einige Herausforderungen.

Ein wesentlicher Aspekt betrifft die Genauigkeit der Roboterarme bei der Anfahrt zu einem auf der Bildebene bestimmten Zielpunkt, sowie den Zugriff auf die N

adel und den Faden. In den durchgeführten Tests zeigten sich dabei geringe Abweichungen zwischen dem berechneten Sollpunkt und der tatsächlich angefahrenen Position, sowie beim Nadel und Faden Zugriff. Diese Abweichungen lassen sich vermutlich auf Messfehler oder Unschärfen in der Kamerakalibrierung zurückführen. Eine präzisere Kalibrierung, sowie eine Optimierung der verwendeten Algorithmen könnten dazu beitragen, diese Abweichungen zukünftig weiter zu minimieren und die Genauigkeit des Gesamtsystems zu verbessern.

Ein weiterer zentraler Aspekt betrifft die elliptische Bewegung der Nadelspitze innerhalb der Wunde. Die Testergebnisse zeigen, dass die Bewegung zwar grundsätzlich entlang einer elliptischen Bahn erfolgt, jedoch Abweichungen am Austrittspunkt auftreten. Hierbei zeigten die Tests eine durchschnittliche Abweichung von bis zu 1 cm vom angestrebten Austrittspunkt, welcher im Idealfall 1 mm hinter der Wunde liegen sollte.

Diese Diskrepanz lässt sich unter anderem auf die gewählte Anfangsausrichtung des TCP zurückführen. Eine präzisere Festlegung der Startorientierung könnte dabei helfen, den gewünschten Austrittspunkt verlässlicher zu treffen und so die Genauigkeit der Nahtführung zu verbessern.

Ein weiterer wesentlicher Faktor ist die Sicherheit des Systems. Der UR3e Roboter bietet integrierte Sicherheitsschutzvorrichtungen, die die Bewegung des UR3e von außerhalb des definierten Arbeitsfelds verhindern.

Trotz der aufgezeigten Herausforderungen konnte die grundsätzliche Geeignetheit des Systems bestätigt werden. Mögliche Ansätze zur weiteren Optimierung werden im folgenden Abschnitt erläutert.

5.2 Lösungsansätze und Weiterentwicklung

Basierend auf den im vorherigen Kapitel dargestellten Herausforderungen ergeben sich verschiedene Ansätze zur Weiterentwicklung des Systems. Im Folgenden werden zentrale Ideen skizziert, die sowohl die Genauigkeit der Positionierung als auch die Robustheit der Bildverarbeitung verbessern könnten:

Eine präzisere Kalibrierung des Roboters und der Koordinatentransformation würde dazu beitragen, die Genauigkeit bei der Bestimmung der Nadelposition zu erhöhen. Der Einsatz automatisierter Kalibrierungsmethoden, z. B. mit Hilfe von Kalibrierungs-Gittern, könnte hierbei hilfreich sein. Für eine präzise Koordinatentransformation könnten fortschrittliche Filtermethoden wie Kalman- oder Partikelfilter implementiert werden. Diese würden helfen, das Sensorrauschen zu minimieren und präzisere Positionsdaten zu erhalten.

Durch die automatische Anpassung der Parameter der Ellipsenbahn (a , b , Orientierung) auf Basis der Wundposition könnte die Bewegung innerhalb der Wunde weiter verfeinert werden. Auch der Einsatz von *Reinforcement Learning* bietet eine vielversprechende Möglichkeit zur Optimierung der elliptischen Bewegung. Durch die Implementierung eines lernbasierten Verfahrens zur Bahnplanung könnte der Roboter in die Lage versetzt werden, die gewünschte Bewegung zunächst selbstständig zu erlernen und anschließend auszuführen.

Bezüglich der Sicherheitsmechanismen könnte eine adaptive Begrenzung der Bewegungen entwickelt werden, die sich dynamisch an die aktuelle Situation anpasst. Beispielsweise könnten sicherheitskritische Bereiche durch eine Kombination aus visueller Erkennung und vordefinierten Arbeitsbereichen flexibel angepasst werden, um den Bewegungsfreiraum des Roboters zu erweitern, ohne die Sicherheit zu gefährden.

Kapitel 6

Fazit

Zusammenfassung

Im Rahmen dieser Bachelorarbeit wurde ein System zur kontinuierlichen Trajektoriensteuerung und zur kamerabasierten Nadelerkennung entwickelt und experimentell erprobt. Ziel war es, die ruckartige, diskontinuierliche Bewegung eines chirurgischen Roboters bei Nadeleinstich und Nadelaustritt durch eine kontinuierliche elliptische Bewegung zu ersetzen und die Position der Nadelspitze automatisiert zu erkennen und zu verfolgen.

Hierfür wurde zunächst ein mathematisches Modell für die elliptische Bewegung aufgestellt und in die Robotersteuerung integriert. Gleichzeitig kam ein bildverarbeitungsbasiertes Verfahren zum Einsatz, das mit Hilfe einer industriellen Kamera sowie ,gezielter Beleuchtung die Position der Nadel in Echtzeit bestimmt. Die Transformation der Bildkoordinaten in das globale Koordinatensystem erfolgt hierbei durch Kalibrierung und gezielte Anwendung linearer Algebra.

Die experimentellen Ergebnisse zeigen, dass die kontinuierliche Bahnführung prinzipiell realisierbar ist. Dennoch traten Herausforderungen auf, insbesondere im Bezug auf Kalibrierungsgenauigkeit, Bildverzerrungen und mechanische Toleranzen beim Nadeleinstich. Auch die exakte Übergabe der Nadel an einen zweiten Roboterarm stellte sich als kritisch heraus, da selbst kleinste Abweichungen im Millimeterbereich signifikante Auswirkungen auf den Erfolg des Greifvorgangs haben können.

Zusammenfassend bietet das entwickelte System eine solide Grundlage für zukünftige Weiterentwicklungen in der robotergestützten Mikrochirurgie. Durch gezielte Optimierungen der Kalibrierung, Bewegungsplanung und visuellen Rückkopplung könnte die Präzision und Zuverlässigkeit des Systems weiter erhöht und somit der Einsatz in medizinischen Anwendungen realistisch unterstützt werden.

Nachwort

Die Arbeit an diesem Projekt war sowohl fachlich anspruchsvoll als auch persönlich bereichernd. Durch die intensive Auseinandersetzung mit den Themen: kontinuierliche Bahnplanung, elliptische Bewegungsmodelle und kameragestützte Nadel- und Fadenerkennung konnte ich meine Kenntnisse in den Bereichen Bildverarbeitung, Robotik und Steuerungstechnik deutlich vertiefen.

Besonders die praktischen Implementierungen und Tests am UR3e-Roboter haben mir verdeutlicht, wie anspruchsvoll die präzise Steuerung von Robotersystemen in sensiblen Anwendungs-

bereichen, wie der chirurgischen Assistenz, ist. Die Kombination aus theoretischen Modellen, softwaretechnischer Umsetzung und experimenteller Validierung stellte eine spannende Herausforderung dar, die mir nicht nur technisches Wissen, sondern auch ein tiefes Verständnis für die Komplexität und Interdisziplinarität dieses Forschungsfeldes vermittelt hat.

Mein Dank gilt meinem Betreuer sowie allen, die mich durch fachliche Hinweise, konstruktives Feedback und tatkräftiger Unterstützung begleitet haben. Ich hoffe, dass die in dieser Arbeit entwickelten Ansätze und gewonnenen Erkenntnisse zur Weiterentwicklung von präzisen, visuell gesteuerten Robotersystemen beitragen und als Grundlage für zukünftige Forschungen in diesem Bereich dienen können.

Literatur

- [1] "Robotik: Definition, Einsatzgebiete, Zukunft.-Zugriff am: 10.Mai 2025. [Online]. Verfügbar: <https://www.iph-hannover.de/de/dienstleistungen/automatisierungstechnik/robotik/#teil>
 - [2] "Robotik: Definition, Einsatzgebiete, Zukunft. Zugriff am: 10.Mai 2025. [Online]. Verfügbar: <https://www.iph-hannover.de/de/dienstleistungen/automatisierungstechnik/robotik/#teil>
 - [3] "Kollaborierende Roboter - Cobots | Universal Robots- Zugriff am: 10 Mai 2025. [Online.] Verfügbar: <https://www.universal-robots.com/de/produkte/ur3-roboter/>
 - [4] Wikipedia. "QRIO.SZugriff am: 19. July 2025. [Online.] Verfügbar: <https://en.wikipedia.org/w/index.php?title=QRIO&oldid=1206716405>
 - [5] "Visual Servoing Architecture of Mobile Manipulators for Precise Industrial Operations on Moving Objects"[Online] Verfügbar: <https://www.mdpi.com/2218-6581/13/5/71>
 - [6] "Kollaborierende Roboter - Cobots | Universal RobotsSZugriff am: 10 Mai 2025. [Online.] Verfügbar: <https://www.universal-robots.com/media/1820624/ur3e-product-factsheet-de-web.pdf>
 - [7] UR3e.pngZugriff am 19. July 2025
 - [8] ur3e-product-factsheet-de-web.SZugriff am: 19. Januar 2025. [Online.] Verfügbar: [ur3e-product-factsheet-de-web.](https://www.universal-robots.com/media/1820624/ur3e-product-factsheet-de-web.pdf) "Zugriff am: 3. Januar 2025. [Online.] Verfügbar:
 - [9] Jianwei Zhang zhang@informatik.uni-hamburg.de, Einführung in die Robotik,"[Online]. Verfügbar unter: https://tams.informatik.uni-hamburg.de/lehre/2011ss/vorlesung/Introduction_to_robotics/folien/Vorlesung3_druck2to1.pdf
 - [10] Universal Robots - DH Parameters for calculations of kinematics and dynamics.SZugriff am: 3. Januar 2025. [Online.] Verfügbar: <https://www.universal-robots.com/articles/ur/application-installation/dh-parameters-for-calculations-of-kinematics-and-dynamics/>
 - [11] Prof. Dr.-Ing. FJ Morales Serrano. "Robotik - Foliensatz.SZugriff am: 2. Februar 2025. [Online.] Verfügbar: https://lms.bht-berlin.de/pluginfile.php/2270444/mod_resource/content/1/2024-04-01%20Robotik%20-%20Foliensatz.pdf
-

- [12] Cobots - CHROMOS Industrial. "Robotiq Hand-E - Kollaborierende Roboter (Cobots).SZugriff am: 19 July Januar 2025. [Online.] Verfügbar: <https://www.cobots.ch/produkt/robotiq-hand-e/>
 - [13] phileon, "Hand-E Instruction Manual,"[Online]. Verfügbar unter: https://assets.robotiq.com/website-assets/support_documents/document/Hand-E_Instruction_Manual_e-Series_PDF_20190122.pdf
 - [14] Wikipedia. "Visual Servoring.SZugriff am: 19. July 2025. [Online.] https://en.wikipedia.org/w/index.php?title=Visual_servoing&oldid=1258898147
 - [15] De Luca , A., Robotics 2- Introduction to Visual Servoring [Online] verfügbar : https://www.diag.uniroma1.it/~deluca/rob2_en/17_VisualServoing.pdf
 - [16] Ellipse-Equation, Formular, Properties, Graphing <https://www.cuemath.com/geometry/ellipse/>
 - [17] Wikipedia .Ellipse". Zugriff am: 26.July 2025 [Online] Verfügbar: <https://de.wikipedia.org/wiki/Ellipse>
 - [18] Wikipedia. "Lochkameramodell.SZugriff am: 26. July 2025. [Online.] Verfügbar: <https://de.wikipedia.org/wiki/Lochkamera>
 - [19] Wikipedia . "Lockmeramodell". Zugriff am: 26.July 2025 [Online] Verfügbar: <https://www.researchgate.net/profile/Luis-Ortiz-25/publication/326518096/figure/fig1/AS:650462126698498@1532093664117/Pinhole-Camera-Model-ideal-projection-of-a-3D-object-on-a-2D-image.png>
 - [20] Wikipedia . "Theorie der Strahlensätze". Zugriff am: 26.July 2025 [Online] Verfügbar: <https://de.wikipedia.org/wiki/Strahlensatz>
 - [21] Google Scholar . "Camera Calibration". Zugriff am: 26.July 2025 [Online] Verfügbar: <https://www.mdpi.com/1424-8220/19/2/246>
 - [22] OpenCv . "Camera Calibration". Zugriff am: 26.July 2025 [Online] Verfügbar: https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html
 - [23] "Bildverarbeitung und Objekterkennung computer vision und Medezin, Herbert SüSse, Erik Rodner"
 - [24] "GigR uEye CP Rev.2ßugriff am: 09.08.2025 [Online] verfügbar https://en.ids-imaging.com/files/downloads/gige-vision/manuals/technical-manual-gev-gige-cp_EN.pdf
-